

1 Topic

1.1 Background and Need

Sudden collapses of aging bridges — a recurring social problem

In April 2023, the walkway of the roughly 30-year-old Jeongja Bridge in Bundang collapsed during ordinary pedestrian use after corrosion and damage accumulated in its cantilever members, killing one person and injuring another. In July 2026, a roughly 50 m section of Boritdol Bridge, an over-sea footbridge in Guryongpo, Pohang, fell into the sea — even though it had received a C grade in a recent inspection and had been repaired and reinforced the previous year; accumulated fatigue from wave action is among the suspected causes. In both cases the bridge collapsed before the next inspection even after warning signs had been observed or repairs made, showing that periodic inspection alone cannot track changes in structural condition continuously enough to prevent accidents.



[Fig. 1: Jeongja Bridge collapse site]



[Fig. 2: Boritdol Bridge collapse site]

As in these two cases, the corrosion, cracking and fatigue that accumulate over years in aging bridges change dynamic properties such as stiffness, damping and natural frequency, so they can be observed through vibration-based monitoring. Vibration characteristics, however, change not only with damage but also with environmental variables such as temperature and humidity, making it hard to tell whether an observed change comes from damage or from the environment. Two tasks remain: capturing changes in dynamic properties continuously between inspections, and separating the environmental effect from those changes so that only damage-related anomalies are flagged.

According to the Korea Authority of Land & Infrastructure Safety's 2025 statistics by facility class (classified by size), Korea has 37,543 bridges — 5,392 Class 1, 7,925 Class 2 and 24,226 Class 3 — and the relatively small Class 3 bridges account for about 64.5%. Because so many structures must be managed, the need for low-cost, continuous screening that also works on small and medium-sized bridges keeps growing.



[Fig. 3: Share of Class 3 bridges among all bridges in Korea]

The continuous-monitoring blind spot of small and medium bridges

Independently of its size, a bridge's condition is rated by safety grade (A-E), and statutory inspections are divided into the precision safety inspection, which checks and evaluates a facility's current condition, and the precision safety diagnosis, a deeper stage that verifies structural safety through engineering analysis and even evaluates load-carrying capacity (Fig. 4). From December 2025, precision safety diagnosis was

extended to aging and low-grade (C·D·E) facilities as well. Precision safety diagnosis, however, is costly and labor-intensive and is performed only on a fixed cycle — every 4 to 6 years depending on the safety grade — so a time gap remains between inspections, and changes in dynamic properties that accumulate during that gap are hard to detect. As a result, aging small and medium bridges such as pedestrian overpasses and small footbridges lack a low-cost supplementary means of continuously detecting changes in condition between inspections.

종류\안전등급	A등급	B·C등급	D·E등급
정기안전점검	반기 1회 이상	반기 1회 이상	1년 3회 이상
정밀안전진단	6년 1회 이상	5년 1회 이상	4년 1회 이상

[Fig. 4: Inspection cycles and targets by facility grade]

No low-cost screening technology before precision diagnosis

Existing SHM (Structural Health Monitoring) systems install sensors on a structure permanently for continuous measurement, but they rely on precision equipment and dedicated infrastructure — ultra-precise piezoelectric accelerometers, high-performance data-acquisition hardware, distributed fiber-optic sensors (e.g., KRISS BOCCA) — and are therefore applied mainly to a few large, critical bridges such as Incheon Bridge; their cost and installation constraints make them impractical to deploy widely on small and medium aging bridges in service. Between periodic precision diagnosis and costly continuous measurement, low-cost continuous screening that precedes precision diagnosis is effectively missing. Bridge safety management therefore needs a multi-stage concept in which continuous screening (stage-1 triage) filters out anomaly candidates before precision safety inspection and diagnosis, so that limited inspection staff and budgets can be concentrated on suspect bridges.

1.2 Topic and Purpose

This work addresses the three problems summarized above: ① capturing changes in dynamic properties continuously between inspections while separating environmental effects, ② the time gap between precision-diagnosis cycles, and ③ the difficulty of applying precision measurement systems to small and medium aging bridges. It is not a system that predicts collapse or replaces precision safety diagnosis; it is a low-cost stage-1 (supplementary) screening system that fills the time gap between inspections and diagnoses by indicating which bridges should be inspected first.

The purpose of this work is to develop a low-cost, machine-learning-based bridge vibration screening system that addresses this blind spot and this missing technology at the same time, with three goals. First, vibration data are collected with an ADXL355Z MEMS accelerometer and a Raspberry Pi (3 Model A+ in stage 1, 5 in stage 2); because damage data cannot be created deliberately on a bridge in service, unsupervised learning trained only on normal data is used. Second, since a bridge's natural frequency rises and falls with air temperature and humidity even without damage [2], DHT22 and DS18B20 sensors record air temperature, surface temperature and relative humidity, and the normal temperature/humidity-natural-frequency relationship is learned so that variation explained by the environment is removed and false alarms are reduced. Third, because an anomaly signal alone does not tell an inspector where to look, the system implements explainable alarms that also show which frequency bands deviated from the normal baseline, guiding the direction of the follow-up precision inspection.

Approach

The three goals are implemented as a single unsupervised anomaly-detection pipeline. 1) Only the vibration patterns of the normal state are learned; 2) after normal variation explained by temperature and humidity is removed first, a residual that deviates strongly from the learned pattern is judged an anomaly candidate; and 3) the system visualizes which frequency bands have a large reconstruction error (the difference between the input and its reconstruction from the learned normal pattern, which grows as the input departs from normal — see 2.3), providing interpretive information that supports the choice of inspection direction rather than a definitive diagnosis. The hardware is described in 2.2, and the concrete procedure that links the three functions is covered step by step in 2.3 Operating Principle.

1.3 Similar Products (Research) and Differentiation

SHM systems based on low-cost MEMS accelerometers

An SHM study combined a Raspberry Pi with a low-cost MEMS accelerometer (ADXL355Z) and showed that a structure's natural frequencies, mode shapes and damping can be identified without ultra-precise piezoelectric sensors costing tens of millions of won [1]. That study, however, focuses on identifying dynamic characteristics and does not address how environmental variables such as temperature and humidity, or changes in operational loading, affect anomaly detection.

Distributed fiber-optic sensor systems

The BOCDA-based fiber-optic sensor system of the Korea Research Institute of Standards and Science (KRISS) precisely measures strain and load position in a structure with high spatial resolution [4]. As precision strain measurement that presupposes dedicated infrastructure, it differs from this work's vibration-based continuous screening in both the physical quantity measured and its purpose.

Unsupervised-learning SHM algorithms

The Swiss Z24 Bridge Benchmark (provided by KU Leuven) is a representative public dataset that includes long-term monitoring and progressive damage-test data [3]. Many unsupervised models — Autoencoder, One-Class SVM, Isolation Forest, VAE and others — have been compared on Z24 [5][6], but most stop at outputting an anomaly score or a threshold exceedance and do not explain which band departed from the baseline.

Environmental-compensation research on long-term data from a full-scale bridge

The openLAB Research Bridge public dataset, from the long-term monitoring of a full-scale research bridge in Bautzen, Germany, provides acceleration responses together with environmental measurements such as air temperature and humidity, making it well suited to analyzing seasonal and daily temperature effects (details in 2.7) [8]. In this work, openLAB serves as supplementary validation data for checking whether the environmental-compensation principle also holds on full-scale long-term data; the core contribution lies in a low-cost hardware implementation that links collection, compensation, judgment and explanation in one pipeline, and in validation with our own model bridges, which distinguishes it from studies that reanalyze public data.

The distinctiveness of this work therefore rests on three axes: direct low-cost hardware implementation, temperature/humidity compensation, and band-wise explainability. Unlike low-cost MEMS studies, it reduces false alarms through temperature/humidity compensation; unlike fiber-optic measurement, it performs continuous screening with inexpensive general-purpose parts; and unlike unsupervised studies that output only an anomaly score, it also reports the contribution of each band to the reconstruction error.

2 Building the System

2.1 Objective

This system is a supplementary vibration-screening system that collects vibration, temperature and relative humidity with a low-cost MEMS accelerometer (ADXL355Z) and a temperature/humidity sensor (DHT22), reduces environment-driven false alarms using the temperature/humidity-frequency relationship learned in the normal state, detects anomaly candidates that depart from the normal pattern with an Autoencoder, and reports them together with the frequency bands that have large reconstruction errors. Our own experiments proceed in three stages. Stage 1 confirms that the whole collection-compensation-detection-explanation pipeline works, using a foam-board model and manual excitation. Stage 2 overcomes stage 1's two limitations (indistinct resonance peaks and variation in manual excitation) with an aluminum multi-span model and a solenoid auto-tapper, validates discrimination performance on a matrix of temperature, humidity, mass and bolt-loosening conditions, and builds a real-time measurement and analysis setup based on a Raspberry Pi 5. Stage 3 moves the system to two real pedestrian overpasses to examine environmental effects and detection power (details in 2.2 and 3.1).

2.2 Design

Hardware

The system is built as a hardware acquisition unit and a software analysis unit. The hardware components and the reasons for each design choice are listed in the table below.

Component	Role · design rationale
ADXL355Z 3-axis accelerometer	3-axis vibration — detects changes in natural frequency and other vibration traits
DHT22 temperature/humidity sensor [7]	Temperature/humidity — compensates for environment-driven normal variation
Raspberry Pi 3 Model A+ (stage 1)	Data collection and storage (Python logging)
Raspberry Pi 5 (stage 2)	Runs real-time acquisition, tapper control, FFT, anomaly score and dashboard on one device
Foam-board bridge model (stage 1)	First model for structural-condition tests such as mass distribution
Aluminum bridge model (stage 2)	Low damping gives sharp resonance peaks; bolted joints give repeatable boundary conditions
Solenoid auto-tapper (stage 2)	Automatic excitation at fixed force, period and position — removes manual variation and narrows normal scatter
DS18B20 surface-temperature sensor (stage 3)	Deck surface temperature — logged as a candidate compensation input (waterproof probe, ₩4,200)
LTE router (stage 3)	Remote access to the field node and data checks (Eduplayer ELR22, ₩59,650)
Power bank (stage 3)	Power for installation where no mains supply exists (30,000 mAh, ₩20,900)

The core measurement and processing hardware — ADXL355Z evaluation board ₩84,589, DHT22 sensor ₩5,680 and Raspberry Pi 5 (about ₩100,000 for the 2-4 GB RAM model) — costs about ₩190,000 in total; adding the DS18B20 surface-temperature sensor (₩4,200), the LTE router (₩59,650) and the power bank (₩20,900) used in stage 3 brings it to about ₩275,000. This is far below existing SHM measurement costing tens of millions of won, which presupposes ultra-precise piezoelectric accelerometers and dedicated acquisition equipment (see 1.3 and [1]), supporting this work's goal of a low-cost configuration that can be deployed on many small and medium bridges.

Software

The software analysis unit consists of two programs, one for acquisition and one for analysis. The acquisition program runs in Python on the Raspberry Pi and writes the ADXL355Z 3-axis acceleration at 200 Hz (500 Hz in stages 2 and 3) and the DHT22 temperature and relative humidity at about 2-second intervals to a single CSV file. The analysis program splits this file into time windows and performs, in order, FFT and feature extraction, residual calculation by multivariate regression on temperature T and relative humidity RH , standardization, reconstruction-error-based anomaly scoring, a 99th-percentile threshold decision, and band-contribution output.

In stage 2 this configuration is extended into a real-time system on a Raspberry Pi 5. The acquisition logger, tapper control and analysis server run on the same device to guarantee a shared clock; the window right after each tap (from $t_0+0.05$ s for 2.0 s, where t_0 is the tap time) is cut out, and its spectrum and anomaly score are computed immediately and shown on a web dashboard. The dashboard refreshes about every 0.7 s with the live z-axis waveform, tap spectrum, ratio to threshold (anomaly score $\div$ threshold), peak frequency, RMS, temperature/humidity and band contributions; a ratio below 0.8 is shown as normal, 0.8-1.0 as caution, and 1.0 or above as an anomaly candidate. The real-time decision is for on-site monitoring; all raw data, including condition codes, are saved to CSV and used for model training and evaluation.

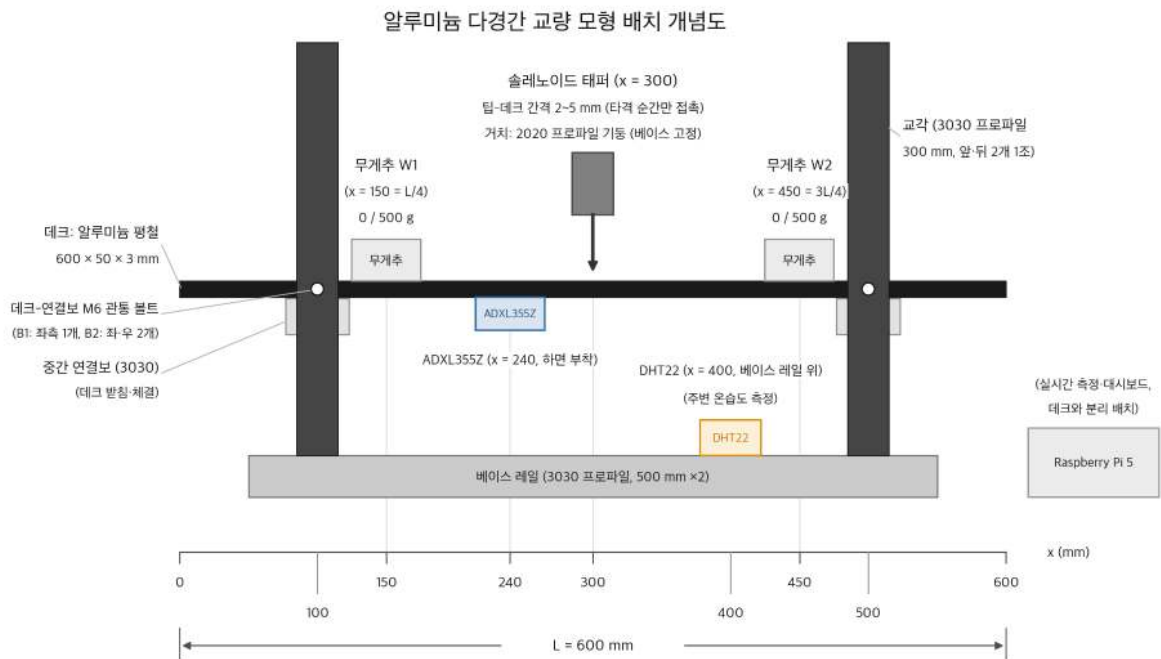
For the stage-3 real bridges, the acquisition program was changed to record ambient vibration continuously without a tapper, the DS18B20 surface temperature was added, and CSV files are split hourly for long unattended collection. Anomaly detection is not run in real time on site but in batch after collection.

Sensor placement

On the stage-1 foam-board model, the ADXL355Z is mounted at mid-span, where the first bending response is largest ($x = L/2$, $y = B/2$ for span L and width B), and the vertical z axis, which reflects vertical bending, is the main axis of analysis.

On the stage-2 aluminum model, every position is defined along the length of the deck (overall length 600 mm) with $x = 0$ at the left end. The solenoid tapper strikes the centerline of the top surface at mid-span, $x = 300$ mm, so that every tap consistently excites the dominant mode, and the gap between tip and deck is held at 2-5 mm with a card gauge in all conditions so that they touch only at the moment of impact. The ADXL355Z is attached to the underside of the deck at $x = 240$ mm, away from directly below the impact point, to measure the large bending response near mid-span while avoiding the local impact component right after the tap and interference from the weights and the impact point on top. Weights are placed at $x = 150$ mm ($L/4$) and $x = 450$ mm ($3L/4$), the same proportions as in stage 1 (Figure 1).

In both experiments the DHT22 and the Raspberry Pi are kept off the vibrating structure (deck and piers) so that they do not affect the bridge's mass or stiffness. In stage 1 they were placed beside the model with the cables fixed loosely; in stage 2 the DHT22 is fixed on the base rail at $x = 400$ mm so that temperature and relative humidity around the bridge are recorded at the same position in every condition, and the Raspberry Pi sits beside the base, away from the deck, so that cable tension and mass do not influence the vibration response.



[Figure 1: Layout of the aluminum bridge model (stage 2)]

2.3 Operating Principle

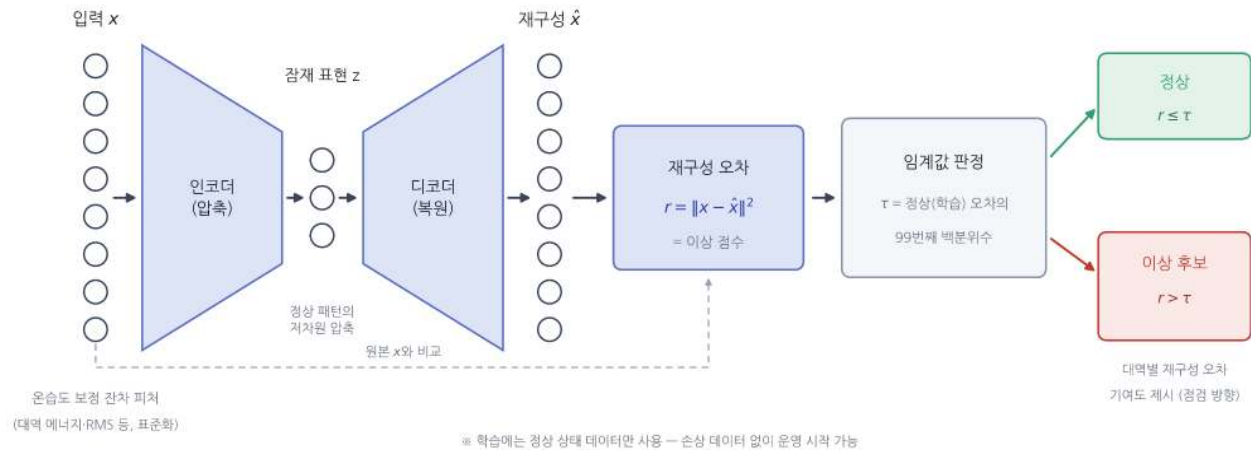
1. Vibration and temperature/humidity acquisition: 3-axis acceleration is collected from the ADXL355Z, and temperature T and relative humidity RH from the DHT22. The actual natural-frequency range was checked in a preliminary test so that the analysis band stays within the Nyquist limit, and the plan was to raise the sampling rate to 500 Hz if the measured fundamental approached or exceeded the 100 Hz Nyquist limit of 200 Hz sampling. On the stage-2 aluminum model, preliminary measurement after assembly showed the dominant-mode frequency above that limit, so the sampling rate was fixed at 500 Hz and the measurement range at ± 8 g (measured natural frequencies are given in 3.4). Excitation is by manual tapping in stage 1 and, in stage 2,

by automatic taps from the solenoid auto-tapper (at a fixed period after warm-up); tap times are logged on the same clock, and only the free-vibration segment right after each tap is used as the unit of analysis.

2. Signal preprocessing and frequency-band feature extraction: the DC offset is removed from the acceleration signal and the spectrum is estimated (FFT/STFT in stage 1; in stage 2 the Welch method, which reduces noise by averaging spectra over short segments). RMS, standard deviation, peak-to-peak, peak frequency and power, band energies and so on are computed per segment and axis; the analysis bands are reset to match the measured dominant peak, and from stage 2 onward eight z-axis features are used, with amplitude and energy log-scaled and band energies normalized as ratios (bands and features are listed in 3.2). Temperature T and relative humidity RH are used for environmental compensation. The z-axis peak frequency serves as the indicator of the dominant-mode natural frequency, and the band-wise analysis is an auxiliary indicator that explains which vibration characteristic departed from the baseline, not a way to locate damage.

3. Multivariate regression compensation of the normal natural frequency by temperature and humidity: on normal data, a multivariate regression that predicts the natural frequency from temperature T and relative humidity RH is learned; at inference, the difference between the measurement f and the prediction $\hat{f} = g(T, RH)$, $r = f - \hat{f}$, is defined as the residual corrected for the temperature/humidity effect. The same regression and residual calculation are applied to every vibration feature, and the resulting residual features are the input to the anomaly-detection model. Because the residual can still contain sensor noise, mounting condition and operational loading besides structural anomaly candidates, it is used as a criterion for detecting anomaly candidates, not as a confirmation of damage.

4. Unsupervised anomaly detection with an Autoencoder: the Autoencoder is trained only on standardized residual features from the normal state (architecture in Figure 2). The model compresses the input into a low-dimensional latent representation and reconstructs it, learning only the correlation structure of normal data, so data that depart from the normal pattern have a larger reconstruction error. The threshold is set at the 99th percentile of the reconstruction error on normal validation data (in stage 2, the error distribution of 90 normal windows held out exclusively for the threshold; see 3.2). Because both training and thresholding use only normal data, operation can begin without any damage data. The stage-1 implementation used the PCA reconstruction error, equivalent to a linear Autoencoder; stage 2 adopted a nonlinear Autoencoder with an 8-3-8 hidden structure (tanh, early stopping).



[Figure 2: Autoencoder-based anomaly detection]

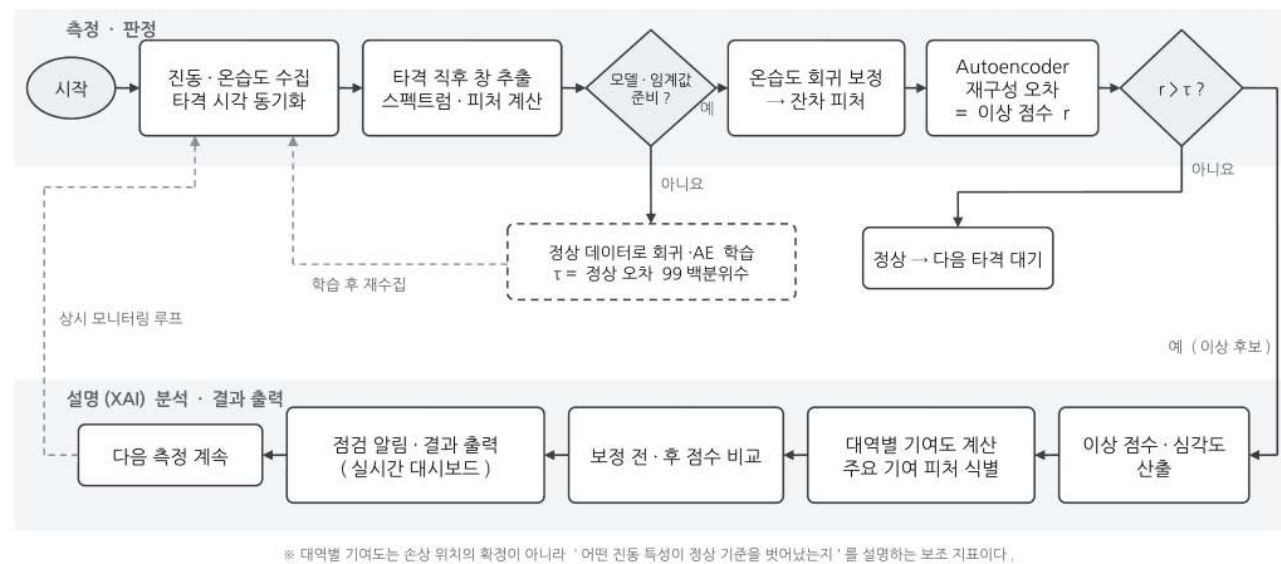
5. Band-wise reconstruction-error contribution: when an anomaly is detected, the system also analyzes which frequency bands (or bands around modes) have large reconstruction errors. A contribution is computed as “that feature's reconstruction error ÷ total reconstruction error.”

6. Output and precision-inspection alerts: when the anomaly score exceeds the threshold, the system raises an alarm and outputs the anomaly score, a before/after comparison of temperature/humidity compensation, the main contributing bands, the rate of change in natural frequency, the band contributions and an interpretation for inspectors. Separately from whether the threshold is exceeded (detection), the ratio to threshold (2.2) and the share of recent windows exceeding the threshold are computed as severity indicators and presented as grounds for prioritizing precision inspection, but

because these indicators express the degree of departure from the normal state, not the physical severity of damage, they are not used to compare different damage types (validation plan in 3.3).

2.4 System Flowchart

The operating structure that links these modules as software is shown in the system flowchart below (Figure 3). Acquisition → preprocessing and feature extraction → environmental compensation → anomaly scoring is the continuous path repeated for every analysis window; only when the anomaly score exceeds the threshold does the flow branch into band-contribution analysis and the precision-inspection alert. Each block corresponds, in order, to steps 1-6 of 2.3.



[Figure 3: Flowchart of the bridge vibration anomaly detection system]

2.5 Development Process

Step	Work	Output
Step 1 (06/03)	Checked sensor links (ADXL355Z, DHT22, Pi 3A+); built the foam-board model and chose sensor positions	Connection test results, mounting photos
Step 2 (06/03-06/04)	Collected normal (0-0) and mass-condition (1-0, 1-1) data (notation in 3.1)	Raw CSV per condition
Step 3 (06/04)	Collected environmental conditions (outdoor temperature excluded); organized 9 conditions	9-condition dataset (baseline, humidity, temp+humidity)
Step 4 (06/04-06/05)	FFT/STFT feature extraction; Autoencoder training and anomaly scoring	Feature dataset, detection results
Step 5 (06/05-06/12)	Band-contribution analysis; stage-1 results summarized	Contribution plots, stage-1 report
Step 6 (06/17-07/08)	Purchased and sourced stage-2 parts (aluminum model, tapper)	All parts
Step 7 (07/11-07/12)	Assembled the model (rail-piers-beams-deck); tapper circuit and Pi 5 real-time dashboard implemented	Stage-2 model and build photos
Step 8 (07/13-07/18)	Normal baseline collection and 27-condition matrix experiment	CSV per condition, tap log
Step 9 (07/19-07/26)	Stage-2 analysis, model comparison, report update	Result tables, plots, final report
Step 10 (09/07-09/25)	Stage-3 field measurement on real bridges E and C; added-mass (100/200 kg) test	CSV per session, analysis results

2.6 Components Built and Key Implementation

Hardware build

In stage 1, an ADXL355Z-DHT22-Raspberry Pi acquisition setup was built on a foam-board model about 55 cm long to verify the concept of temperature/humidity-compensated anomaly detection. Because damping and assembly variation in the foam board made the resonance peaks broad, stage 2 extended the same sensing and compensation setup to an aluminum multi-span model.



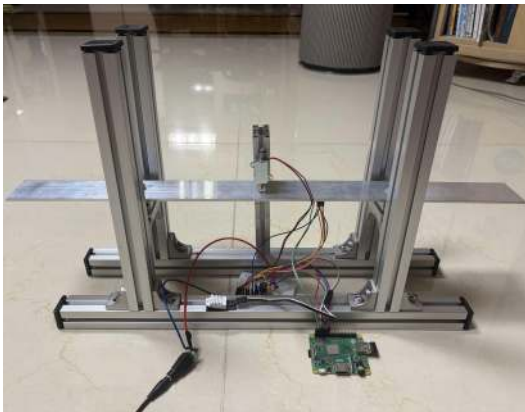
Foam-board bridge model for stage 1 (about 55 cm long)



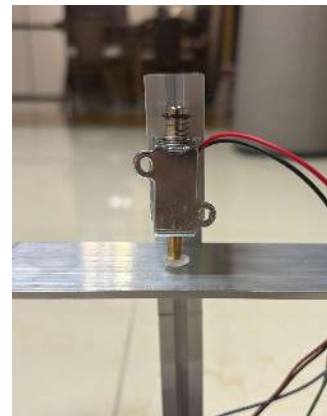
ADXL355Z 3-axis MEMS accelerometer



DHT22 temperature/humidity sensor



Aluminum bridge model for stage 2 (about 60 cm long)



Solenoid auto-tapper



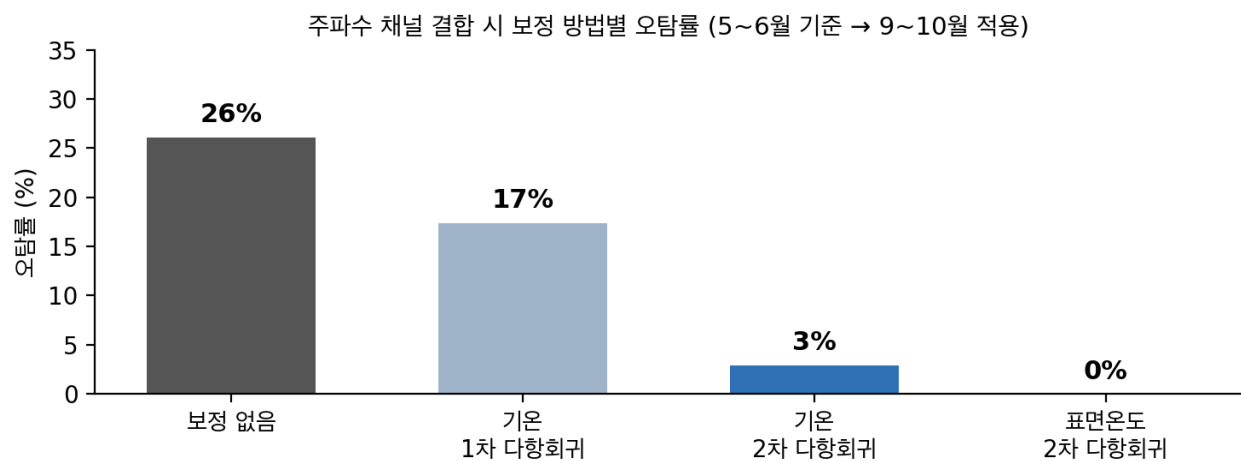
Raspberry Pi 5 real-time dashboard

2.7 Validation on Full-Scale openLAB Data

How temperature compensation behaves over seasonal change when a frequency-decision channel is combined with the autoencoder was checked with the openLAB dataset published by TU Dresden [8]. The structure is a three-span concrete research bridge about 45 m long; 526 records of air temperature, humidity and acceleration from spring to autumn 2024 are provided, and because the bridge was undamaged throughout, every alarm is a false alarm. In each

record, all three natural frequencies rose as the temperature fell (about $-0.19\%/^{\circ}\text{C}$ for the 7.05 Hz mode), and the rise steepened below 15°C , reaching 6-8% in October. The normal baseline and threshold were set from summer (May-June) records and applied unchanged to 69 autumn records (mid-September to October) to compare false-alarm rates by compensation method. The decision combined the current 8-feature autoencoder with a frequency channel that checks whether the compensated frequency of the 7.05 Hz mode — the mode most affected by temperature — falls outside the central 99% range of the summer records (0.5% excluded at each end); an alarm was counted if either channel was exceeded. Both channels use 99% of normal records as their reference.

Compensation (combined decision)	False alarms: training (May-Jun, 184)	False alarms: evaluation (Sep-Oct, 69)	Evaluation records colder than the training minimum temperature (40)
No compensation	1.6%	26.1%	42.5%
Air temp., 1st-order polynomial	1.6%	17.4%	27.5%
Air temp., 2nd-order polynomial	1.6%	2.9%	2.5%
Surface temp., 2nd-order polynomial	1.6%	0.0%	0.0%



[Figure 4: False-alarm rates by compensation method on 69 openLAB records from Sep-Oct with the frequency channel combined (summer baseline applied unchanged)]

With the original autoencoder alone, the anomaly score stayed at about $0.1 \times$ the threshold in October even though the frequency had risen by 6-8%, so it did not see the seasonal change at all. With the frequency channel combined and no compensation, 26% of the autumn records were judged anomalous, reproducing the surge in uncompensated false alarms reported in prior work [2] (Figure 4). Compensation with a first-order polynomial in air temperature lowered the false-alarm rate only to 17%, because a linear relationship learned over the summer cannot extrapolate the nonlinear rise at low temperatures. With second-order polynomial compensation the rate fell to 3%, and with second-order compensation on surface temperature to 0%, back to the level of the training period (about 2%). Relative humidity left the rate unchanged when added to the second-order air-temperature model (3→3%) and was therefore excluded, consistent with prior findings [2] that natural-frequency variation is explained mainly by temperature.

3 Testing

3.1 Test Purpose and Experimental Design

The tests have three purposes. First, to examine with low-cost sensors (2.2) and model bridges how changes in structural condition affect the vibration features and anomaly scores. Second, to verify the reduction in false alarms from temperature/humidity compensation; the full-scale comparison is handled by openLAB (2.7). Third, to move the validated system to two real pedestrian overpasses and examine environmental effects on bridge E and detection power on bridge C (stage 3). The core proof of performance is stage 2, where conditions can be controlled; stage 3 is supplementary validation in the field, where no damage labels exist.

Stage-1 design — foam-board model and mass conditions

100 g weights at L/4 and 3L/4 create three conditions: 0-0 (normal), 1-0 (left) and 1-1 (both sides).

Stage-2 design — aluminum multi-span model and a 27-condition matrix

The factors below are run as a matrix of 3 environment states × 3 mass levels × 3 bolt levels = 27 conditions. Because the heater changes temperature and humidity at the same time, temperature alone cannot be changed; the environment therefore has three states: no change (Tx-Hx), humidity change (Tx-Ho) and temperature/humidity change (To-Ho). This design yields interaction data between environment (T-H) and simulated damage (M-B), and tests the value of temperature/humidity compensation, which should classify environmental change as normal and damage as anomalous.

Factor (code)	Levels	How applied
Environment (T-H)	No change / humidity / temp + humidity (3 levels)	Tx-Hx = no environmental input. Tx-Ho = high humidity in an opaque box with a humidifier (temperature held). To-Ho = heater raises temperature and lowers humidity. Analysis uses DHT22 measurements
Mass change (M)	0 / 500 / 1,000 g (3 levels)	At two marks, x = 150 and 450 mm: M500 = 500 g on one side (asymmetric); M1000 = 500 g on each side (symmetric). 50 g weights stacked in the same order every time
Bolt loosening (B)	0 / 1 / 2 bolts (3 levels)	Deck-beam through-bolts loosened by 1/2 turn from fully tightened. B1 = one left bolt (asymmetric); B2 = left and right (symmetric boundary damage)

Loosening is limited to the deck-beam through-bolts, which act directly on the deck's boundary conditions, and B1 is a subset of B2 so that boundary stiffness is lowered step by step. After the bolt conditions, the original tightening is restored to check that the anomaly score returns to baseline. The normal class is defined as the three M0-B0 cells (three environment states), so that a state in which only the environment changed is learned as normal; because these cells are used for training and thresholding, they are collected longer than the other conditions.

The runs are grouped into three environment blocks, each containing the 9 mass × bolt conditions. Each condition receives 10 warm-up taps (not recorded) followed by 35 recorded taps; each of the three normal cells receives 180 recorded taps. The normal cells are split in time order into 120 training, 30 threshold and 30 evaluation windows, giving 360 training, 90 threshold and 90 evaluation windows in total.

Stage-3 design — field measurement on real bridges

Ambient vibration was measured for 30 minutes on each of five candidate overpasses (A-E) and compared as in the table below. E (1.74 Hz) and C (3.81 Hz), whose dominant peaks stand out more than 10,000-fold over the band median, were selected; D, with no visible peak, A, with too little vibration, and B, with impulsive vibration mixed in, were excluded. E, where several days of observation were possible, was assigned to environmental validation, and C, where loading was possible, to detection-power validation.

Bridge	Dominant peak	Peak clarity (vs band median)	z-axis variation (mg, 1σ)	Decision
E	1.74 Hz	17,369×	1.36	Selected (1st)
C	3.81 Hz	17,632×	2.57	Selected (2nd)
B	3.69 Hz	1,698×	7.27	Excluded (impulsive vibration)
A	2.62 Hz	194×	0.43	Excluded (vibration too small)
D	No peak	3.3×	0.81	Excluded (no identifiable peak)

Excitation is the ambient vibration from pedestrians and vehicles, and a deck surface-temperature sensor (DS18B20) was added to the single node. Bridge E, assigned to environmental validation, was recorded in 7 sessions over 4 days, September 7-10, for about 68 hours. Because damage cannot be inflicted on a real bridge, recall is not computed; the normal state is assumed on the basis of the most recent inspection grade, and threshold exceedances are treated as false alarms. The main indicators are the mode

peak reproducibility, the false-alarm rate when one session's threshold is applied unchanged to the other sessions, and the temperature sensitivity of the peak frequency. Bridge C, assigned to detection power, was used for a reversible added-mass test; preliminary measurements in three sessions with the same mounting on September 20 (about 6 hours) identified the peak to track. The required weight follows from the frequency being proportional to $\sqrt{\text{stiffness/mass}}$. If M_{eq} is the equivalent mass that acts effectively on the mode at the loading position, a weight m much smaller than this lowers the frequency by a fraction of about $m/2M_{\text{eq}}$, so the weight needed for a target reduction δ well above the measurement error is $m \approx 2\delta M_{\text{eq}}$. Since M_{eq} cannot be known without drawings, a small weight m_p permitted on site is loaded first, $M_{\text{eq}} \approx m_p / ((f_0/f_p)^2 - 1)$ is back-calculated from the unloaded and loaded frequencies f_0 and f_p , and the main-test mass is then chosen.

The stage-3 acquisition program and logging intervals are as in 2.2, without condition codes. Each session's raw CSV has columns for timestamp, DHT22 air temperature and relative humidity, DS18B20 surface temperature, ADXL355Z internal temperature and 3-axis acceleration in g, and is identified by the session start time in the folder name; mounting, remounting and traffic events are kept in field notes and linked to windows during analysis.

3.2 Test Method

Testing is divided into analysis of the public openLAB data (2.7), our own model-bridge experiments and field measurements on real bridges. Because openLAB is undamaged throughout, it is used to compare false-alarm rates across compensation methods; our own experiments verify low-cost sensor acquisition, detection of mass change and reproduction of compensation. Only the differences between experiments are described below.

In stage 1, 0-0, 1-0 and 1-1 were excited manually and recorded repeatedly for at least 20-30 seconds per condition (3 environmental conditions, 4-second windows). The models are trained only on normal 0-0 data, and the threshold — the 99th percentile of the normal reconstruction error — is applied identically to the three models (uncompensated, temperature, temperature/humidity). All three use the PCA reconstruction of 2.3 (equivalent to a linear AE).

The key change in stage 2 is replacing manual excitation with automatic taps from the solenoid auto-tapper, which narrows the scatter of the normal baseline. Tap times and acceleration are synchronized on the same clock, and only the window right after each tap is analyzed.

Data are collected at 500 Hz sampling with a ± 8 g range, and the analysis window is fixed at 2.0 s (1,000 samples) starting at $t_0 + 0.05$ s to exclude the impact. The spectrum is estimated with Welch's method, and the analysis bands were reset to 0-40, 40-80 and 80-125 Hz to match the measured dominant mode (101 Hz). The features are eight z-axis quantities (log RMS, log p2p, peak frequency, log peak power, log total energy and three band-energy ratios); amplitudes are log-scaled and band energies normalized as ratios so that changes in spectral structure, not tap strength, stand out.

Training, thresholding and evaluation are kept separate to prevent data leakage. Training uses 360 normal windows; the threshold is the 99th percentile of the reconstruction error on 90 dedicated windows not used in training; evaluation uses 90 normal and 840 damage windows. The model is fixed as a nonlinear Autoencoder with an 8-3-8 hidden structure (tanh), only the environmental inputs are varied (uncompensated, temperature, temperature/humidity), and runs are repeated with 5 random seeds to check stability.

In stage 3, the z-axis signal collected at 500 Hz and ± 2 g is low-pass filtered at 20 Hz and downsampled to 50 Hz, and the same eight features as in stage 2 are computed with only the analysis bands reset below 20 Hz to match each bridge's dominant peak; on both bridges a 120-second window is moved in 10-second steps. The 4-second window used in the model experiments was sized for short tap records and is too short for state changes on a real bridge that last tens of minutes. For bridge E, the model is trained and its threshold set in one session and then applied unchanged to other sessions with the same mounting (also checked across remounted sessions), comparing four models: uncompensated, air temperature, air temperature + humidity, and with surface temperature added. Decisions are based on exceedances in consecutive windows, not a single window, to prevent false alarms from single events such as several people crossing at once. The alarm model for bridge C, matched to the loading test and its excitation conditions, is trained on the first 20 minutes of 30 minutes of unloaded walking at the same location on September 25 and its false-alarm rate checked on the last 10 minutes. The detection test back-calculated M_{eq} from a 100 kg preliminary load, set the main-test mass at 200 kg, and proceeded as unloaded walking → loaded walking for 30 min → loaded heel drops (excitation by dropping the heels) → heel drops after removal (with little traffic at night, the experimenter

provided the excitation by walking). Alarms are decided by applying the model trained on unloaded walking unchanged, and the cause is judged from the drop and recovery of the frequency of the vibration that dies away on its own after a heel drop (free decay).

3.3 Expected Results

In stage 1, added mass is expected to lower the peak frequency, and in the asymmetric condition (1-0) changes in amplitude and band energy are expected to appear first. In stage 2, because the weights sit near the supports and their effective mass is small, the dominant-mode frequency should fall clearly but by less than the mass ratio, and bolt loosening should bring a small drop in natural frequency along with harmonics and wider peaks. The main question is whether the anomaly score rises monotonically as conditions within the same damage type become more severe (M0 → M500 → M1000, B0 → B1 → B2); this ordering must hold for the anomaly score to be usable as a severity indicator. In stage 3, as in prior work [2], the false-alarm rate of the uncompensated model is expected to grow in sessions at different temperatures and the compensated model to reduce it, although early-autumn observations do not include the cold range below 15 °C, so the effect may be small. In the added-mass test, the natural frequency is expected to drop with mass and recover after removal, with changes of about 0.4% or more distinguishable.

3.4 Actual Results

For the stage-1 foam-board model, the analysis bands were set to 0-12, 12-25 and 25-50 Hz to match the fundamental-mode peak in the preliminary measurement (about 22.6 Hz). Under loading on both sides (1-1) the peak frequency fell by 8.9%; under loading on one side (1-0) the peak change stayed within the scatter, but the multi-feature anomaly score detected it and the threshold-exceedance rate rose to 29.7%. In the humidity-change segment, temperature/humidity compensation reduced the false-alarm rate from 17.5% to 9.1%, and over the whole evaluation segment the compensated models were also lower than the uncompensated one (9.2% → temperature 4.8%, temperature/humidity 6.3%). The recall of the temperature/humidity-compensated model, however, was only 0.234, missing most damage windows.

Stage-2 results

The temperature/humidity-compensated model detected all 840 damage windows (recall 100%) and raised false alarms on only 4 of 90 normal evaluation windows (4.4%; ROC-AUC 0.996), all four in the simultaneous temperature/humidity (To·Ho) block (4/30 in the table below). The dominant mode of the normal baseline is 101.0 Hz, and each damage group in the table sums the three environment blocks (105 windows each). The peak shifts are far larger than expected in 3.3 (e.g., -64.4%) and even reverse (M1000·B1 -19.8%) not because the natural frequency actually fell, but because the peak frequency, which reads only the single tallest peak, jumped from the 101 Hz mode to another mode as damage changed the relative strength of the modes (mode switching, 3.5). The -10.9% in the normal heated block (To·Ho) is likewise attributed to mode switching caused by heating, not to an actual loss of stiffness.

Condition group	Median peak frequency	Peak shift (vs 101 Hz)	Mean anomaly score	Exceedance rate
Normal (Tx·Hx)	101.0 Hz	+0.0%	0.03	0/30 (0%)
Humidity change (Tx·Ho)	101.0 Hz	+0.0%	0.07	0/30 (0%)
Temp + humidity (To·Ho)	90.0 Hz	-10.9%	4.13	4/30 (13.3%)
M500·B0	36.0 Hz	-64.4%	33.2	105/105 (100%)
M1000·B0	36.0 Hz	-64.4%	128.4	105/105 (100%)
M0·B1	64.0 Hz	-36.6%	264.7	105/105 (100%)
M0·B2	40.0 Hz	-60.4%	370.3	105/105 (100%)
M500·B1	44.0 Hz	-56.4%	75.4	105/105 (100%)
M1000·B1	81.0 Hz	-19.8%	108.3	105/105 (100%)
M500·B2	66.0 Hz	-34.7%	307.1	105/105 (100%)

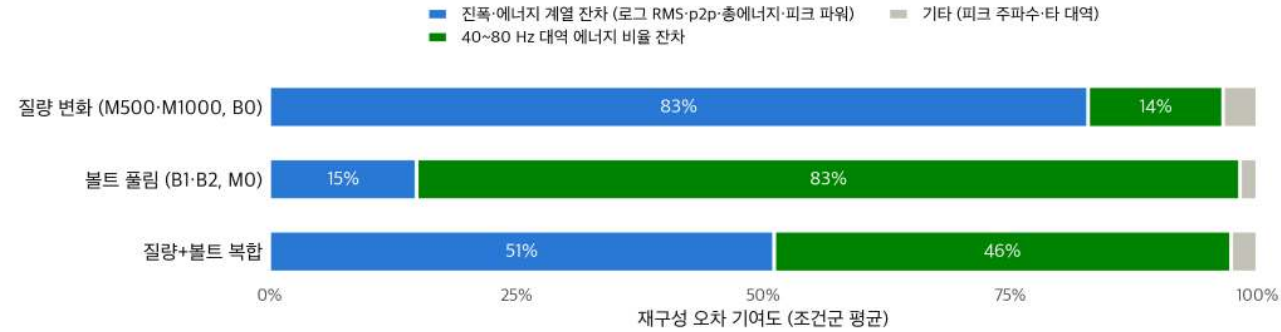
M1000·B2	53.0 Hz	-47.5%	323.4	105/105 (100%)
----------	---------	--------	-------	----------------

Model	False-alarm rate	Precision	Recall	F1-score
Uncompensated AE (no environmental input)	0.022	0.998	0.993	0.995
Temperature-compensated AE (T residual)	0.033	0.996	1.000	0.998
Temp/humidity-compensated AE (T·RH residual)	0.044	0.995	1.000	0.998

In stage 2, false alarms of the compensated model rose from 2 to 4, while damage recall stayed at 100% regardless of the seed (interpretation in 3.5).

Metric	Uncompensated AE	Temp/humidity-compensated AE
False-alarm rate, humidity block (Tx·Ho)	0.0% (0/30)	0.0% (0/30)
False-alarm rate, temp+humidity block (To·Ho)	6.7% (2/30)	13.3% (4/30)
Damage recall (840 windows, range over 5 seeds)	96.2~99.3%	100% (all seeds)

The 2 versus 4 in To·Ho is a difference of two out of 30 windows — small enough to arise by chance given the few windows — and 3 of the 4 compensated false alarms are borderline exceedances at 1.0-1.5× the threshold. Contributions by damage type are shown in Figure 5.



[Figure 5: Feature contributions to reconstruction error by damage type — mass, bolt and combined profiles]

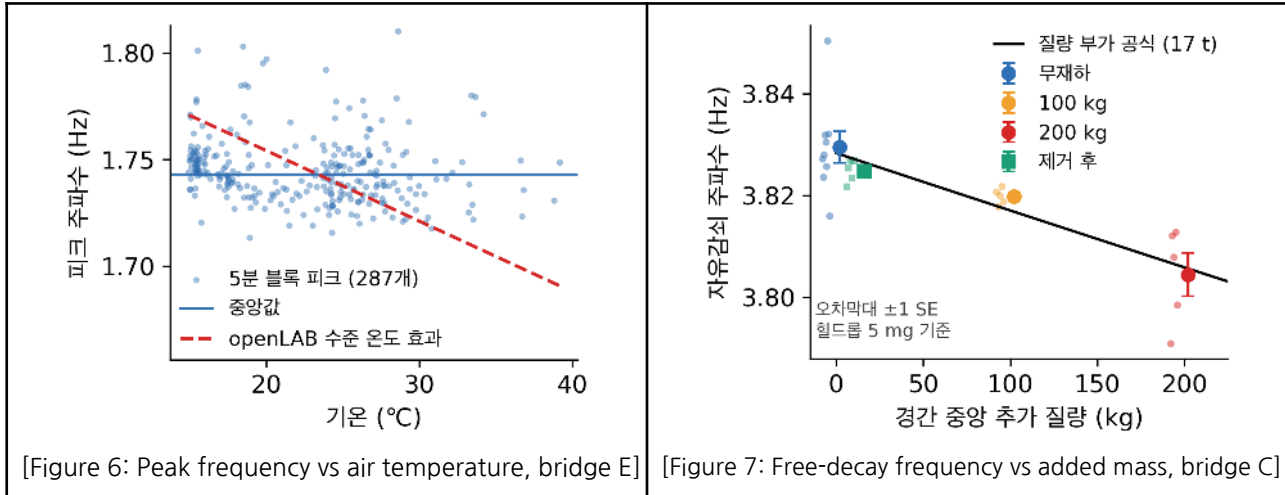
Stage-3 results

On bridge E, the peak at about 1.74 Hz stood clearly above the noise and was tracked stably in every session over the four days, with differences between sessions below 1%. The uncompensated false-alarm rate when one session's threshold was applied to another stayed mostly at or below 2%, even after remounting.

No benefit from temperature/humidity compensation appeared. When the temperature ranges of training and evaluation overlapped, all four models had the same false-alarm rate of about 1-2%, and when the evaluation temperature fell outside the training range, the compensated models' false-alarm rates actually increased. Controlling for traffic amplitude within the same mounting period (air temperature about 15-39 °C), the estimated temperature slope of the peak frequency was indistinguishable from zero. With an openLAB-level temperature effect ($-0.19\%/^{\circ}\text{C}$), the peak should have dropped by about 4.5% over the 24 °C range, but the measurements were flat throughout (Figure 6). The peak change implied by the measured slope was about 0.5%, at the level of the fluctuation between 5-minute blocks. An amplitude dependence — the peak falling slightly as traffic amplitude rose — was more pronounced.

The exceeding windows did not come from temperature either. Exceedances concentrated in the evening and late night, when traffic was light, especially in low-amplitude windows, and a sensor error that logs zeros on all three axes occurred intermittently on both bridges (about 10% of E's records; those windows were excluded from analysis). In the preliminary measurement on bridge C (three sessions with the same mounting on September 20, about 6 hours), the 3.81 Hz mode peak chosen in the screening measurement was reproduced in every session, and because an adjacent peak also appeared, the tracking rule was fixed in advance. The frequency drop under the 100 kg preliminary load was barely distinguishable from the measurement error, so the main-test mass was set at 200 kg. With 200 kg placed at mid-span, the frequency

fell by 0.6% and returned to its original value after removal; after correcting for amplitude differences, the frequencies with no load, 100 kg and 200 kg lay within error on a single added-mass curve, consistent with an equivalent mass of about 17 t (Figure 7). The model trained on 20 minutes of unloaded walking exceeded the threshold on 18% of windows in the 10 minutes of unloaded walking not used for training, but with no exceedance lasting three or more consecutive windows; during 30 minutes of walking with 200 kg loaded, 46% of windows exceeded and about 32% led to consecutive-3-window alarms (at most 7 consecutive windows). On a single-window basis the separation is weak, since 18% exceeded even without load, but the consecutive-exceedance rate separated the two clearly: the consecutive-window rule filtered out false alarms without missing the change in state. Whether an alarm was caused by mass was established from the drop and recovery of the natural frequency.



3.5 Analysis of Results

The stage-1 results matched the expected physics. The 8.9% drop in peak frequency for 1-1 is exactly what is expected from the natural frequency falling as mass increases ($f = (1/2\pi)\sqrt{(k/m)}$), and for 1-0 the amplitude and band features responded, raising the exceedance rate from 4.5% (normal) to 29.7% and confirming the need for an anomaly score that combines several features. However, the damping of the foam board and the variation of manual tapping made the scatter of normal measurements large, and these two issues became the improvement targets of stage 2.

In stage 2, first, standardizing the excitation with a machine fixed the normal baseline: the peaks of all 60 room-temperature normal evaluation windows coincided at 101 Hz within the frequency resolution. Second, the 27 conditions were separated almost completely. The median ratio to threshold was

0.003-0.038× for normal windows and 5.8-73.7× for damage windows, a gap of more than a hundredfold. However, one normal window in the heated block spiked to 15.9× and the weakest damage window reached only 2.0×, so in operation the decision is based on the exceedance rate over consecutive windows. The severity ordering held for mass, with a monotonic increase from M500 5.8× to M1000 22.6× in all three blocks; for bolts alone the score grew from B1 to B2, but combined with mass the ordering reversed (M500-B1 75.4 < M0-B1 264.7), making it hard to use as a severity indicator. Third, the composition of reconstruction-error contributions points to the type of damage (Figure 5). In bolt-loosening windows the 40-80 Hz band-ratio residual accounted for 76-90%, in mass windows the amplitude/energy features for 73-80%, and in combined windows the two groups appeared side by side at 51% and 46%. If the band ratio dominates, the boundary condition (bolts) should be inspected first; if amplitude/energy dominate, the change in load. The stage-1 conclusion that the peak frequency is unstable under mode switching and that features must be combined remained valid. Fourth, the normal training data already covered 22.4-39.9 °C and 48.9-79.0% humidity, so false alarms were rare even without compensation, and the effect of compensation was limited to maintaining recall (100% for all 5 seeds versus 96.2-99.3% uncompensated). The remaining 4 false alarms are windows near the hottest point, 39.9 °C — a limitation of approximating the structure's temperature with air temperature.

The largest share of the performance gain came from feature normalization. Because amplitudes differed by up to about 40× between conditions, raw features gave a recall of only 21.3% (0% for mass windows), whereas log and ratio normalization raised it to 100% on the same data.

The stage-3 results show four things. First, the system still works when moved to real bridges: on both overpasses the mode peaks were reproduced within 1% and false alarms were low, showing that visit-style inspection with a node moved from bridge to bridge is feasible. Second, the seasonal surge in false alarms reported in prior work [2] did not recur on bridge E because the conditions differed. Z24 used, directly in its decision, a natural frequency that rose sharply as the asphalt layer froze during a year of observation including sub-zero periods. Bridge E differed in both environment and decision method. Environmentally, it was four early-autumn days at 15 °C and above with no frost, so the temperature-driven peak change was only about 0.5%, at the level of the fluctuation between blocks (3.4). In the decision, the autoencoder does not look at frequency directly but compares a score built from eight features with a 99th-percentile threshold, so a frequency shift of a few percent barely shows in the score. In the low-temperature extrapolation of openLAB, whose conditions resemble Z24, the surge reappeared when the frequency channel was combined and was resolved by compensation (2.7). Compensation is therefore needed when the environmental change exceeds the feature scatter and the decision looks at frequency directly, and inputs outside the training range add extrapolation error and should only be flagged. Third, the cause of false alarms on the real bridges was not temperature but excitation conditions such as low amplitude late at night. Bridge E's exceeding windows were dominated by the amplitude/energy group, so the contributions also pointed to a change in excitation, whereas in bridge C's loaded segment the three groups were split evenly and the contributions alone could not identify the cause. Fourth, the frequency drop and recovery under both loads (100 kg and 200 kg) agree with the added-mass formula, so tracking the natural frequency provides the basis for identifying the cause of an alarm.

4 Interpretation & Conclusion

4.1 Conclusion

Using only low-cost commercial parts (ADXL355Z, DHT22, Raspberry Pi), this system separated the 27-condition matrix of a laboratory model. The temperature/humidity-compensated model detected all 840 damage windows and raised false alarms on only 4 of 90 normal evaluation windows (recall 100%, false-alarm rate 4.4%, ROC-AUC 0.996), resolving the missed detections of stage 1, where recall was only 0.234, through standardized excitation and feature normalization.

Environmental compensation reduced false alarms when training did not cover the environmental variation (stage-1 humidity segment 17.5% → 9.1%; openLAB seasonal extrapolation with the frequency channel combined 26% → 0-3%), with the effect coming mainly from temperature; in stage 2, where the environmental range was covered in advance, it appeared only as maintained recall. In the contribution analysis, the 40-80 Hz band-ratio residual dominated for bolt loosening and the amplitude/energy residuals for mass change, so the alarm alone narrowed the direction of inspection.

On the stage-3 real bridges, the mode peaks of both overpasses were tracked within 1% with few false alarms; in the early-autumn field, however, the temperature dependence of the peak was below the noise, so no compensation benefit was confirmed, and false alarms came from insufficient excitation and traffic events. In the added-mass test, a 0.6% frequency drop and recovery under 200 kg was measured, showing that a single low-cost node can distinguish a mass change of about 1% of the mode's equivalent mass (about 17 t). Based on these results, field operation decides with 120-second windows and a rule of three consecutive exceedances, excludes low-amplitude periods without traffic from decisions, and automatically reinitializes the sensor if an error persists for more than a few seconds.

4.2 Future Improvements

Four tasks remain. First, finalize how contributions are reported on real bridges. Because real bridges have no damage labels and the amplitude/energy features depend on traffic volume, the stage-2 correspondence will not be used as is; instead, changes relative to a normal period will be reported by feature group (amplitude, band structure, frequency). Causes will be presented only as candidate starting points for inspection, without judging location or extent (presence detection, level 1 of Rytter's four levels of damage identification [9]), and the contribution profile for each candidate will first be validated on the Z24 data, which include a recorded damage history [3]. Second, re-validate the effect of temperature/humidity compensation by observing for several weeks or more with the same mounting to include sub-zero periods and seasonal transitions, splitting training and evaluation days by date, and securing environmental inputs by shading the air-temperature sensor and pressing the surface-temperature sensor firmly against the deck. Third, integrate the frequency-decision channel (2.7) into the real-time system. Fourth, extend to multiple nodes to estimate the location of damage.

References

- [1] Caballero-Russi, D., Ortiz, A. R., Guzmán, A., & Canchila, C., “Design and Validation of a Low-Cost Structural Health Monitoring System for Dynamic Characterization of Structures,” *Applied Sciences*, 12(6), 2807, 2022.
- [2] Peeters, B., & De Roeck, G., “One-year monitoring of the Z24-Bridge: environmental effects versus damage events,” *Earthquake Engineering & Structural Dynamics*, 30(2), 149-171, 2001.
- [3] Z24 Bridge Benchmark documentation, KU Leuven, Leuven, Belgium.
- [4] Korea Research Institute of Standards and Science (KRISS), distributed fiber-optic sensor technology based on BOCDA (Brillouin Optical Correlation Domain Analysis).
- [5] Giglioni, V., Venanzi, I., Baia, A. E., Poggioni, V., Milani, A., & Ubertini, F., “Deep Autoencoders for Unsupervised Damage Detection with Application to the Z24 Benchmark Bridge,” *European Workshop on Structural Health Monitoring (EWSHM 2022)*, *Lecture Notes in Civil Engineering*, Springer, 2023. DOI: 10.1007/978-3-031-07258-1_105
- [6] Pakzad, S. S., & Masoodi, A. R., “Early damage detection in bridges using a variational autoencoder-based hybrid unsupervised learning framework,” *Scientific Reports*, 16, 1389, 2026. DOI: 10.1038/s41598-025-31115-w
- [7] Aosong Electronics, DHT22 (AM2302) temperature/humidity sensor datasheet.
- [8] Jansen, A., Richter, B., Röder, R., Herbers, M., & Marx, S., "Monitoring Data of the openLAB Research Bridge (2024-02-01 to 2024-10-31) and building information," *OPARA - Open Access Repository and Archive*, TU Dresden, 2025. DOI: 10.25532/OPARA-660.
- [9] Rytter, A., "Vibrational Based Inspection of Civil Engineering Structures," PhD thesis, Aalborg University, Denmark, 1993.

List of Materials

No.	Type	Label in text	Description (keywords)	Notes (source, etc.)
1	Photo	Material 1	Jeongja Bridge collapse site photo	Safety Journal report (2023.4.7), https://www.anjunj.com/news/articleView.html?idxno=36589
2	Photo	Material 2	Boritdol Bridge collapse photo (CCTV still)	Yonhap News report (2026.7.28), screen capture, https://v.daum.net/v/20260728161255607
3	Chart	Material 3	Share of Class 3 bridges among all bridges in Korea	Drawn by the authors from the Korea Authority of Land & Infrastructure Safety's 2025 facility statistics
4	Table	Material 4	Inspection cycles and targets by facility grade	Enforcement Decree of the Special Act on the Safety and Maintenance of Facilities, Annex 3 (amended 2025.12.4); drawn by the authors

5	도식·그래프	그림1~그림7	배치 개념도, Autoencoder 구조, 시스템 순서도, 실험 결과 그래프	직접 제작
6	사진	사진 일체	모형 교량, 완성 모듈, 2차 모형 제작 과정 사진	직접 촬영
7	공개 데이터	데이터셋	openLAB(TU Dresden, OPARA — 2.7 분석에 사용), Z24 Bridge Benchmark(KU Leuven — 선행 연구 비교 참조)	공개 데이터, 참고문헌 3·8
8	오픈소스 SW	-	Python, NumPy, pandas, scikit-learn, Matplotlib (수집·분석·시각화)	각 공식 배포본(PSF·BSD 라이선스)

소스코드

sentry.py

```

from __future__ import annotations
import argparse
import csv
import io
import json
import math
import socket
import re
import tempfile
import subprocess
import sys
import threading
import time
from collections import deque
from pathlib import Path

import numpy as np
import pandas as pd
from flask import Flask, jsonify

HTML = r'''<!doctype html>
<html lang="ko">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>SENTRY 보안 대시보드</title>
<style>
body{margin:0;background:#10141b;color:#eef2f7;font-family:Arial,sans-serif}.wrap{
max-width:1100px;margin:auto;padding:20px}.top{display:flex;justify-content:space-
between;align-items:center;gap:16px;margin-bottom:16px}.title{font-size:28px;font-
weight:700}.small{font-size:12px;color:#9ca8b8;margin-top:7px}.status{padding:11px
18px;border-radius:999px;font-weight:700;white-space:nowrap}.normal{background:#17
6b45}.warning{background:#916b12}.anomaly{background:#9d2c2c}.waiting{background:#
465164}.grid{display:grid;grid-template-columns:repeat(6,1fr);gap:12px}.card{backg
round:#1a202b;border-radius:14px;padding:16px;box-shadow:0 6px 18px
#0004}.full{grid-column:span 6}.wide{grid-column:span

```

```

3}.label{font-size:13px;color:#9ca8b8}.value{font-size:27px;font-weight:700;margin-top:6px}.buttons{display:flex;gap:10px;flex-wrap:wrap}.button{border:0;border-radius:10px;padding:11px 15px;font-weight:700;color:white;background:#2f73bf;cursor:pointer}.button.warn{background:#8b6518}.button.stop{background:#6b3440}.progress{height:10px;background:#111722;border-radius:999px;overflow:hidden;margin-top:12px}.progress>div{height:100%;width:0;background:#6db7ff}.alarmbox{border:1px solid #344052}.alarmbox.active{border-color:#d64b4b;box-shadow:0 0 28px #d64b4b55}canvas{width:100%;height:280px;background:#111722;border-radius:10px}.thresholds{display:grid;grid-template-columns:repeat(3,1fr);gap:12px;margin-top:12px}.threshold{background:#111722;border-radius:10px;padding:12px}.reason{font-size:18px;font-weight:700;margin-top:8px}@media(max-width:800px){.grid{grid-template-columns:repeat(2,1fr)}.wide,.full{grid-column:span 2}.top{align-items:flex-start;flex-direction:column}.thresholds{grid-template-columns:1fr}}
</style>
</head>
<body>
<div class="wrap">
<div class="top"><div><div class="title">SENTRY 기체 보안 대시보드</div><div id="connection" class="small">서버 연결 확인 중</div></div><div id="securityStatus" class="status waiting">감시 해제</div></div>
<div class="grid">
<div class="card"><div class="label">진동 RMS</div><div id="vibration" class="value">-</div><div id="vibrationRatio" class="small">-</div></div>
<div class="card"><div class="label">순간 피크</div><div id="peakMotion" class="value">-</div><div id="peakRatio" class="small">-</div></div>
<div class="card"><div class="label">자세 변화</div><div id="tilt" class="value">-</div><div id="tiltThreshold" class="small">-</div></div>
<div class="card"><div class="label">표면온도 · DS18B20</div><div id="temp" class="value">-</div><div id="surfaceState" class="small">대기</div></div>
<div class="card"><div class="label">습도 · DHT22</div><div id="hum" class="value">-</div><div id="humidityState" class="small">대기</div></div>
<div class="card"><div class="label">누적 경보</div><div id="alertCount" class="value">0</div></div>
<div class="card full alarmbox" id="alarmBox"><div class="label">최근 경보</div><div id="reason" class="reason">없음</div><div id="lastAlert" class="small">-</div></div>
<div class="card full"><div class="label">CSV 저장 상태</div><div id="storage" class="small">대기</div></div>
<div class="card full"><div class="label">최근 3축 합성 진동</div><canvas id="wave"></canvas></div>
<div class="card full"><div class="label">감시 기준</div><div class="thresholds"><div class="threshold"><div class="label">진동 기준</div><div id="vibrationThreshold" class="value">-</div></div><div class="threshold"><div class="label">충격 기준</div><div id="peakThreshold" class="value">-</div></div><div class="threshold"><div class="label">자세 기준</div><div id="angleThreshold" class="value">-</div></div></div><div class="progress"><div id="calibrationBar"></div></div><div id="calibrationText" class="small">기체를 설치한 뒤 감시 시작을 누른다.</div></div>
<div class="card full"><div class="buttons"><button class="button" onclick="action('/api/security/arm')">감시 시작 · 기준 재설정</button><button class="button warn" onclick="action('/api/security/reset')">경보 해제</button><button class="button stop" onclick="action('/api/security/disarm')">감시 해제</button></div><div id="info" class="small">-</div></div>
</div>
</div>
<script>
let audioContext=null,lastAlert=false,lastConnected=true,lastArmed=false;
function
enableAudio(){if(!audioContext)audioContext=new(window.AudioContext||window.WebKitAudioContext)();if(audioContext.state==='suspended')audioContext.resume();if('Noti

```

```

fication'in
window&&Notification.permission==='default')Notification.requestPermission() }
function beep() {if(!audioContext)return;const
o=audioContext.createOscillator(),g=audioContext.createGain();o.frequency.value=88
0;g.gain.setValueAtTime(.16,audioContext.currentTime);g.gain.exponentialRampToValu
eAtTime(.001,audioContext.currentTime+.28);o.connect(g);g.connect(audioContext.des
tination);o.start();o.stop(audioContext.currentTime+.3)}
function notify(title,body){if('Notification'in
window&&Notification.permission==='granted')new Notification(title,{body})}
function fmt(v,d,u){return v==null?'-':Number(v).toFixed(d)+u}
function line(canvas,x,y){const
c=document.getElementById(canvas),d=devicePixelRatio||1,w=c.clientWidth,h=c.client
Height;c.width=w*d;c.height=h*d;const
g=c.getContext('2d');g.scale(d,d);g.clearRect(0,0,w,h);g.strokeStyle='#2c3544';g.b
eginPath();for(let i=1;i<5;i++){const
yy=h*i/5;g.moveTo(0,yy);g.lineTo(w,yy)}g.stroke();if(!y||y.length<2)return;let
ymin=Math.min(...y),ymax=Math.max(...y);if(ymax===ymin){ymax+=1;ymin-=1}g.strokeSt
yle='#67b2ff';g.lineWidth=2;g.beginPath();for(let i=0;i<y.length;i++){const
px=i/(y.length-1)*w,py=h-(y[i]-ymin)/(ymax-ymin)*h;if(i===0)g.moveTo(px,py);else
g.lineTo(px,py)}g.stroke()}
async function action(path){enableAudio();try{await
fetch(path,{method:'POST'})};await update()}catch(e){}}
async function update(){const conn=document.getElementById('connection');try{const
r=await fetch('/api',{cache:'no-store'});if(!r.ok)throw new Error('api');const
s=await r.json();const
st=document.getElementById('securityStatus');st.textContent=s.security_status_text
||'대기 중';st.className='status
'+(s.security_status_class||'waiting');document.getElementById('vibration').textCo
ntent=fmt(s.vibration_rms_g,4,'
g');document.getElementById('peakMotion').textContent=fmt(s.vibration_peak_g,4,'
g');document.getElementById('tilt').textContent=fmt(s.orientation_change_deg,1,'°'
);document.getElementById('tiltThreshold').textContent=s.orientation_threshold_deg
==null?'-': '기준
'+s.orientation_threshold_deg.toFixed(1)+'°';document.getElementById('temp').textC
ontent=fmt(s.surface_temp_c,1,'
°C');document.getElementById('hum').textContent=fmt(s.humidity,1,'
%');document.getElementById('surfaceState').textContent=s.surface_status||'대기';do
cument.getElementById('humidityState').textContent=s.humidity_status||'대기';docume
nt.getElementById('storage').textContent=(s.logger_status||'')+ '
'+(s.storage_status||'')+ '
'+(s.active_csv||'파일 대기')+ '
'+(s.run_folder||'');document.getElementById('alertCount').textContent=String(s.al
ert_count||0);document.getElementById('vibrationThreshold').textContent=fmt(s.vibr
ation_threshold_g,4,'
g');document.getElementById('peakThreshold').textContent=fmt(s.peak_threshold_g,4,'
g');document.getElementById('angleThreshold').textContent=fmt(s.orientation_thresh
old_deg,1,'°');document.getElementById('vibrationRatio').textContent=s.vibration_r
atio==null?'-': '기준의
'+s.vibration_ratio.toFixed(2)+'배';document.getElementById('peakRatio').textConte
nt=s.peak_ratio==null?'-': '기준의 '+s.peak_ratio.toFixed(2)+'배';const
progress=Math.max(0,Math.min(100,s.calibration_progress||0));document.getElementBy
Id('calibrationBar').style.width=progress+'%';document.getElementById('calibration
Text').textContent=s.calibration_text||'-';document.getElementById('reason').textC
ontent=s.last_alert_reason||'없음';document.getElementById('lastAlert').textContent
=s.last_alert_at_text||'-';document.getElementById('info').textContent=s.info||'-';
document.getElementById('alarmBox').className='card full
alarmbox'+(s.security_alert?'
active':'');line('wave',s.wave_x,s.wave_y);if(s.updated_at===null){conn.textContent
='서버 연결됨 · 센서 데이터 대기'}else{const
age=Math.max(0,Date.now()/1000-s.updated_at);conn.textContent=age<3?'서버 연결됨 ·
실시간 감시 중:'서버 연결됨 · 마지막 센서 데이터 '+age.toFixed(1)+'초
전'}if(s.security_alert&&!lastAlert){beep();setTimeout(beep,700);notify('SENTRY
도난·이동 의심',s.last_alert_reason||'기체 움직임이
감지됨')}}lastAlert=!s.security_alert;lastConnected=true;lastArmed=!s.security_arm

```

```

ed}catch(e){conn.textContent='서버 연결 끊김';const
st=document.getElementById('securityStatus');st.textContent='연결
끊김';st.className='status
anomaly';if(lastConnected&&lastArmed){beep();notify('SENTRY 연결 끊김','장치 전원
또는 네트워크 상태를 확인해야 한다.')}lastConnected=false}}
setInterval(update,500);update();
</script>
</body>
</html>'''

```

```

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="SENTRY continuous security
dashboard")
    parser.add_argument("--logger", default="logger.py")
    source = parser.add_mutually_exclusive_group()
    source.add_argument("--bridge", help="기존 CSV 한 개 열기(자동 파일 전환 없음)")
    source.add_argument("--manifest", help="실행 폴더의 current.json을 따라가기")
    parser.add_argument("--condition", default="SECURITY")
    parser.add_argument("--output-dir", default="data/security")
    parser.add_argument("--sample-rate", type=int, choices=[125, 250, 500],
default=125)
    parser.add_argument("--range-g", type=int, choices=[2, 4, 8], default=8)
    parser.add_argument("--no-dht", action="store_true")
    parser.add_argument("--no-surface", action="store_true")
    parser.add_argument("--surface-device", help="DS18B20 장치 ID: 28-...")
    parser.add_argument("--rotate-seconds", type=float, default=3600.0)
    parser.add_argument("--flush-seconds", type=float, default=1.0)
    parser.add_argument("--fsync-seconds", type=float, default=10.0)
    parser.add_argument("--env-stale-seconds", type=float, default=10.0)
    parser.add_argument("--host", default="0.0.0.0")
    parser.add_argument("--port", type=int, default=8080)
    parser.add_argument("--calibration-seconds", type=float, default=20.0)
    parser.add_argument("--window-seconds", type=float, default=0.5)
    parser.add_argument("--step-seconds", type=float, default=0.25)
    parser.add_argument("--vibration-multiplier", type=float, default=4.0)
    parser.add_argument("--peak-multiplier", type=float, default=4.0)
    parser.add_argument("--min-vibration-g", type=float, default=0.015)
    parser.add_argument("--min-peak-g", type=float, default=0.08)
    parser.add_argument("--tilt-deg", type=float, default=8.0)
    parser.add_argument("--tilt-hold-seconds", type=float, default=1.0)
    parser.add_argument("--vote-window", type=int, default=5)
    parser.add_argument("--required-votes", type=int, default=3)
    parser.add_argument("--stale-seconds", type=float, default=5.0)
    parser.add_argument("--auto-arm", action="store_true")
    args = parser.parse_args()
    if not re.fullmatch(r"[\w.-]+", args.condition) or args.condition in {".",
".."}:
        parser.error("condition에는 문자, 숫자, _, -, .만 사용할 수 있습니다.")
    for name in ("rotate_seconds", "flush_seconds", "fsync_seconds",
"env_stale_seconds"):
        if not math.isfinite(getattr(args, name)) or getattr(args, name) <= 0:
            parser.error(f"{name}은 유효한 양수여야 합니다.")
    return args

def metadata_path(bridge: Path) -> Path:
    if not bridge.name.endswith("_bridge_log.csv"):
        raise ValueError("bridge CSV 이름은 *_bridge_log.csv 형식이어야 합니다.")
    return bridge.with_name(bridge.name[:-len("_bridge_log.csv")] +
"_metadata.json")

def metadata_for(bridge: Path) -> dict:

```

```

if bridge.suffix.lower() == ".json":
    return json.loads(bridge.read_text(encoding="utf-8"))
path = metadata_path(bridge)
if path.exists():
    return json.loads(path.read_text(encoding="utf-8"))
return {"sample_rate_hz": 125, "range_g": 8}

def tail_dataframe(path: Path, rows: int) -> pd.DataFrame:
    # CSV 전체를 읽지 않는다. 쓰기 중인 마지막 불완전 행은 다음 조회로 미룬다.
    with path.open("rb") as file:
        header = file.readline().decode("utf-8").strip()
        file.seek(0, 2)
        position = file.tell()
        blocks, count = [], 0
        while position > 0 and count <= rows + 1:
            size = min(65536, position)
            position -= size
            file.seek(position)
            block = file.read(size)
            blocks.append(block)
            count += block.count(b"\n")
        truncated = position > 0
        raw = b"".join(reversed(blocks))
    # 마지막 \n까지의 완성된 레코드만 사용한다.
    raw = raw[:raw.rfind(b"\n") + 1]
    lines = raw.decode("utf-8", errors="ignore").splitlines()
    if lines and lines[0] == header:
        lines = lines[1:]
    elif truncated and lines:
        lines = lines[1:]
    lines = lines[-rows:]
    if not lines or not header:
        return pd.DataFrame()
    return pd.read_csv(io.StringIO(header + "\n" + "\n".join(lines)))

def manifest_csv(source: Path, filename: str | None) -> Path | None:
    if filename is None:
        return None
    if Path(filename).name != filename or filename in {".", ".."}:
        raise ValueError("current.json의 CSV 이름이 올바르지 않습니다.")
    return source.parent / filename

def read_live_tail(source: Path, rows: int) -> tuple[pd.DataFrame, dict, Path]:
    metadata = metadata_for(source)
    if source.suffix.lower() != ".json":
        return tail_dataframe(source, rows), metadata, source
    current = manifest_csv(source, metadata.get("current_csv"))
    if current is None:
        raise ValueError("current.json에 current_csv가 없습니다.")
    data = tail_dataframe(current, rows)
    previous = manifest_csv(source, metadata.get("previous_csv"))
    if len(data) < rows and previous is not None and previous.exists():
        older = tail_dataframe(previous, rows - len(data))
        frames = [frame for frame in (older, data) if not frame.empty]
        data = pd.concat(frames, ignore_index=True) if frames else pd.DataFrame()
    return data.tail(rows), metadata, current

def last_number(data: pd.DataFrame, column: str) -> float | None:
    # dropna().iloc[-1]는 지난 파일/지난 시각의 값을 되살리므로 사용하지 않는다.
    if data.empty or column not in data:

```

```

        return None
    value = pd.to_numeric(data[column].iloc[-1], errors="coerce")
    return finite_or_none(value)

def finite_or_none(value: float | None) -> float | None:
    if value is None:
        return None
    number = float(value)
    return number if math.isfinite(number) else None

def format_alert_time(value: float | None) -> str | None:
    if value is None:
        return None
    return time.strftime("%Y-%m-%d %H:%M:%S", time.localtime(value))

def security_window_metrics(values: np.ndarray) -> tuple[float, float, np.ndarray, np.ndarray]:
    center = np.median(values, axis=0)
    dynamic = values - center
    magnitude = np.linalg.norm(dynamic, axis=1)
    rms = float(np.sqrt(np.mean(magnitude * magnitude)))
    peak = float(np.max(magnitude))
    return rms, peak, center, magnitude

class SecurityMonitor:
    def __init__(self, bridge: Path, args: argparse.Namespace):
        self.bridge = bridge # JSON manifest 또는 단일 CSV
        self.logger_process = getattr(args, "logger_process", None)
        self.env_stale_seconds =
float(metadata_for(bridge).get("env_stale_seconds", args.env_stale_seconds))
        self.recent_data = pd.DataFrame()
        self.fs = float(metadata_for(bridge).get("sample_rate_hz",
args.sample_rate))
        self.calibration_seconds = max(5.0, args.calibration_seconds)
        self.window_seconds = max(0.2, args.window_seconds)
        self.step_seconds = max(0.1, args.step_seconds)
        self.vibration_multiplier = max(1.0, args.vibration_multiplier)
        self.peak_multiplier = max(1.0, args.peak_multiplier)
        self.min_vibration_g = max(0.0001, args.min_vibration_g)
        self.min_peak_g = max(0.0001, args.min_peak_g)
        self.orientation_threshold_deg = max(1.0, args.tilt_deg)
        self.orientation_hold_seconds = max(0.2, args.tilt_hold_seconds)
        self.vote_window = max(1, args.vote_window)
        self.required_votes = min(max(1, args.required_votes), self.vote_window)
        self.stale_seconds = max(2.0, args.stale_seconds)
        self.lock = threading.Lock()
        self.baseline_vector: np.ndarray | None = None
        self.vibration_threshold_g: float | None = None
        self.peak_threshold_g: float | None = None
        self.calibration_start_ns: float | None = None
        self.calibration_requested = False
        self.armed = False
        self.alert_latched = False
        self.alert_count = 0
        self.last_alert_at: float | None = None
        self.last_alert_reason: str | None = None
        self.last_evaluated_ns = 0.0
        self.tilt_started_ns: float | None = None
        self.motion_votes: deque[bool] = deque(maxlen=self.vote_window)
        self.alert_path = (

```

```

        bridge.parent / "security_alerts.csv" if bridge.suffix.lower() ==
".json"
        else bridge.with_name(bridge.name.replace("_bridge_log.csv",
"_security_alerts.csv"))
    )
    self.state = {
        "security_armed": False,
        "security_alert": False,
        "security_status_text": "감시 해제",
        "security_status_class": "waiting",
        "vibration_rms_g": None,
        "vibration_peak_g": None,
        "orientation_change_deg": None,
        "vibration_threshold_g": None,
        "peak_threshold_g": None,
        "orientation_threshold_deg": self.orientation_threshold_deg,
        "vibration_ratio": None,
        "peak_ratio": None,
        "temp_c": None, # 참고용 DHT22 기온. 화면의 주 온도는 표면온도이다.
        "surface_temp_c": None,
        "surface_age_s": None,
        "dht_age_s": None,
        "humidity": None,
        "surface_enabled": True,
        "dht_enabled": True,
        "active_csv": "",
        "run_folder": str(bridge.parent),
        "storage_status": "waiting",
        "rotate_seconds": args.rotate_seconds,
        "alert_count": 0,
        "last_alert_at": None,
        "last_alert_at_text": None,
        "last_alert_reason": None,
        "calibration_progress": 0.0,
        "calibration_text": "기체를 설치한 뒤 감시 시작을 누른다.",
        "wave_x": [],
        "wave_y": [],
        "updated_at": None,
        "info": "센서 데이터를 기다리는 중",
    }
    threading.Thread(target=self.loop, daemon=True).start()
    if args.auto_arm:
        self.arm()

def arm(self) -> dict:
    with self.lock:
        self.calibration_requested = True
        self.calibration_start_ns = None
        self.baseline_vector = None
        self.vibration_threshold_g = None
        self.peak_threshold_g = None
        self.armed = False
        self.alert_latched = False
        self.last_evaluated_ns = 0.0
        self.motion_votes.clear()
        self.tilt_started_ns = None
        self.state.update(
            {
                "security_armed": False,
                "security_alert": False,
                "security_status_text": "보정 중",
                "security_status_class": "warning",
                "calibration_progress": 0.0,
            }
        )

```

```

        "calibration_text": f"{self.calibration_seconds:.0f}초 동안
기체를 건드리지 않는다.",
        "vibration_rms_g": None,
        "vibration_peak_g": None,
        "orientation_change_deg": None,
        "vibration_ratio": None,
        "peak_ratio": None,
    }
)
return dict(self.state)

def disarm(self) -> dict:
    with self.lock:
        self.calibration_requested = False
        self.armed = False
        self.alert_latched = False
        self.motion_votes.clear()
        self.tilt_started_ns = None
        self.state.update(
            {
                "security_armed": False,
                "security_alert": False,
                "security_status_text": "감시 해제",
                "security_status_class": "waiting",
                "calibration_progress": 0.0,
                "calibration_text": "감시가 해제되었다.",
            }
        )
    return dict(self.state)

def reset_alert(self) -> dict:
    with self.lock:
        self.alert_latched = False
        self.motion_votes.clear()
        self.tilt_started_ns = None
        if self.armed:
            text, style = "감시 중", "normal"
        elif self.calibration_requested:
            text, style = "보정 중", "warning"
        else:
            text, style = "감시 해제", "waiting"
        self.state.update(
            {
                "security_alert": False,
                "security_status_text": text,
                "security_status_class": style,
            }
        )
    return dict(self.state)

def calibrate(self, data: pd.DataFrame) -> None:
    values = data[["ax_g", "ay_g", "az_g"]].to_numpy(float)
    window_samples = max(16, int(self.fs * self.window_seconds))
    if len(values) < window_samples * 4:
        return
    rms_values = []
    peak_values = []
    for start in range(0, len(values) - window_samples + 1, window_samples):
        rms, peak, _, _ = security_window_metrics(values[start:start +
window_samples])
        rms_values.append(rms)
        peak_values.append(peak)
    if len(rms_values) < 4:
        return

```

```

        baseline = np.median(values, axis=0)
        vibration_threshold = max(self.min_vibration_g,
float(np.quantile(rms_values, 0.99)) * self.vibration_multiplier)
        peak_threshold = max(self.min_peak_g, float(np.quantile(peak_values,
0.99)) * self.peak_multiplier)
        with self.lock:
            self.baseline_vector = baseline
            self.vibration_threshold_g = vibration_threshold
            self.peak_threshold_g = peak_threshold
            self.calibration_requested = False
            self.armed = True
            self.alert_latched = False
            self.motion_votes.clear()
            self.state.update(
                {
                    "security_armed": True,
                    "security_alert": False,
                    "security_status_text": "감시 중",
                    "security_status_class": "normal",
                    "vibration_threshold_g": vibration_threshold,
                    "peak_threshold_g": peak_threshold,
                    "calibration_progress": 100.0,
                    "calibration_text": "기준 측정이 끝났으며 보안 감시가 시작되었다.",
                    "info": f"기준 RMS {np.median(rms_values):.5f} g | 기준 피크
{np.median(peak_values):.5f} g | 최근 {self.vote_window}개 중
{self.required_votes}개 초과 시 경보",
                }
            )

    def write_alert(self, reason: str, now: float, metrics: dict) -> None:
        new_file = not self.alert_path.exists()
        with self.alert_path.open("a", newline="", encoding="utf-8") as file:
            writer = csv.writer(file)
            if new_file:
                writer.writerow(["t_unix_s", "time", "reason", "vibration_rms_g",
"vibration_peak_g", "orientation_change_deg"])
            writer.writerow(
                [
                    f"{now:.6f}",
                    format_alert_time(now),
                    reason,
                    f"{metrics['vibration_rms_g']:.8f}",
                    f"{metrics['vibration_peak_g']:.8f}",
                    f"{metrics['orientation_change_deg']:.4f}",
                ]
            )

    def trigger_alert(self, reason: str, metrics: dict) -> None:
        with self.lock:
            if self.alert_latched:
                return
            now = time.time()
            self.alert_latched = True
            self.alert_count += 1
            self.last_alert_at = now
            self.last_alert_reason = reason
            self.state.update(
                {
                    "security_alert": True,
                    "security_status_text": "도난·이동 의심",
                    "security_status_class": "anomaly",
                    "alert_count": self.alert_count,
                    "last_alert_at": now,
                    "last_alert_at_text": format_alert_time(now),
                }
            )

```

```

        "last_alert_reason": reason,
    }
)
self.write_alert(reason, now, metrics)

def evaluate(self, data: pd.DataFrame, max_t: float) -> None:
    with self.lock:
        baseline = None if self.baseline_vector is None else
self.baseline_vector.copy()
        vibration_threshold = self.vibration_threshold_g
        peak_threshold = self.peak_threshold_g
        armed = self.armed
        if not armed or baseline is None or vibration_threshold is None or
peak_threshold is None:
            return
        segment = data[data["t_perf_ns"] >= max_t - self.window_seconds * 1e9]
        if len(segment) < max(16, int(self.fs * self.window_seconds * 0.7)):
            return
        values = segment[["ax_g", "ay_g", "az_g"]].to_numpy(float)
        vibration_rms, vibration_peak, current_vector, magnitude =
security_window_metrics(values)
        baseline_norm = float(np.linalg.norm(baseline))
        current_norm = float(np.linalg.norm(current_vector))
        angle = 0.0
        if baseline_norm > 1e-6 and current_norm > 1e-6:
            cosine = float(np.dot(baseline, current_vector) / (baseline_norm *
current_norm))
            angle = math.degrees(math.acos(max(-1.0, min(1.0, cosine))))
        vibration_exceeded = vibration_rms >= vibration_threshold
        peak_exceeded = vibration_peak >= peak_threshold
        motion = vibration_exceeded or peak_exceeded
        self.motion_votes.append(motion)
        votes = sum(self.motion_votes)
        if angle >= self.orientation_threshold_deg:
            if self.tilt_started_ns is None:
                self.tilt_started_ns = max_t
            else:
                self.tilt_started_ns = None
            tilt_sustained = self.tilt_started_ns is not None and (max_t -
self.tilt_started_ns) / 1e9 >= self.orientation_hold_seconds
            reason = None
            if vibration_peak >= peak_threshold * 2.0:
                reason = "강한 충격이 감지됨"
            elif tilt_sustained:
                reason = "기체 자세 변화가 지속됨"
            elif motion and angle >= self.orientation_threshold_deg * 0.5:
                reason = "진동과 자세 변화가 함께 감지됨"
            elif votes >= self.required_votes:
                reason = "진동이 반복적으로 감지됨"
            vibration_ratio = vibration_rms / vibration_threshold
            peak_ratio = vibration_peak / peak_threshold
            recent_t = segment["t_perf_ns"].to_numpy(float)
            metrics = {
                "vibration_rms_g": vibration_rms,
                "vibration_peak_g": vibration_peak,
                "orientation_change_deg": angle,
            }
        with self.lock:
            if self.alert_latched:
                text, style = "도난·이동 의심", "anomaly"
            elif motion or angle >= self.orientation_threshold_deg:
                text, style = "진동 감지", "warning"
            else:
                text, style = "감시 중", "normal"

```

```

self.state.update(
    {
        "security_armed": True,
        "security_alert": self.alert_latched,
        "security_status_text": text,
        "security_status_class": style,
        "vibration_rms_g": vibration_rms,
        "vibration_peak_g": vibration_peak,
        "orientation_change_deg": angle,
        "vibration_threshold_g": vibration_threshold,
        "peak_threshold_g": peak_threshold,
        "orientation_threshold_deg": self.orientation_threshold_deg,
        "vibration_ratio": vibration_ratio,
        "peak_ratio": peak_ratio,
        "wave_x": ((recent_t - recent_t[0]) / 1e9).tolist(),
        "wave_y": magnitude.tolist(),
        "info": f"최근 {self.vote_window}개 판정 중 움직임 {votes}개 |
자세 변화 {angle:.1f}° | {self.alert_path.name}",
    }
)
if reason is not None:
    self.trigger_alert(reason, metrics)

def loop(self) -> None:
    rows = int(self.fs * max(self.calibration_seconds + 5.0, 20.0))
    while True:
        try:
            if not self.bridge.exists():
                time.sleep(0.3)
                continue
            data, metadata, active_csv = read_live_tail(self.bridge, rows)
            with self.lock:
                self.state.update({
                    "active_csv": active_csv.name,
                    "storage_status": metadata.get("status", "CSV 직접 보기"),
                    "rotate_seconds": metadata.get("rotate_seconds", None),
                    "surface_enabled": metadata.get("surface_enabled",
"surface_temp_c" in data),
                    "dht_enabled": metadata.get("dht_enabled", True),
                })
            required = {"t_perf_ns", "ax_g", "ay_g", "az_g"}
            if data.empty or not required.issubset(data.columns):
                time.sleep(0.3)
                continue
            # 파일 경계에서도 기준 보정/최근 과형을 유지한다. 메모리는 최근 rows개로
제한.
            frames = [frame for frame in (self.recent_data, data) if not
frame.empty]
            data = pd.concat(frames, ignore_index=True)
            data = data.drop_duplicates(subset=["t_perf_ns"], keep="last")
            data = data.sort_values("t_perf_ns").tail(rows).copy()
            self.recent_data = data
            for column in ["t_perf_ns", "t_unix_s", "ax_g", "ay_g", "az_g",
"temp_c", "humidity", "surface_temp_c",
"surface_age_s", "dht_age_s"]:
                if column in data.columns:
                    data[column] = pd.to_numeric(data[column],
errors="coerce")
            data = data.dropna(subset=["t_perf_ns", "ax_g", "ay_g",
"az_g"]).sort_values("t_perf_ns")
            if data.empty:
                time.sleep(0.3)
                continue
            max_t = float(data["t_perf_ns"].iloc[-1])

```



```

        except Exception as exc:
            with self.lock:
                self.state["info"] = f"대기/오류: {exc}"
            time.sleep(0.25)

def get(self) -> dict:
    with self.lock:
        result = dict(self.state)
        for key in [
            "vibration_rms_g",
            "vibration_peak_g",
            "orientation_change_deg",
            "vibration_threshold_g",
            "peak_threshold_g",
            "orientation_threshold_deg",
            "vibration_ratio",
            "peak_ratio",
            "temp_c",
            "surface_temp_c",
            "surface_age_s",
            "dht_age_s",
            "humidity",
            "updated_at",
            "last_alert_at",
            "calibration_progress",
        ]:
            result[key] = finite_or_none(result.get(key))
            elapsed = max(0.0, time.time() - result["updated_at"]) if
result.get("updated_at") else math.inf
            for value_col, age_col, enabled_col, status_col in (
                ("surface_temp_c", "surface_age_s", "surface_enabled",
"surface_status"),
                ("humidity", "dht_age_s", "dht_enabled", "humidity_status"),
            ):
                age = result.get(age_col)
                stale = elapsed > self.env_stale_seconds or (
                    age is not None and age + elapsed > self.env_stale_seconds)
                if not result[enabled_col]:
                    result[value_col], result[status_col] = None, "미사용"
                elif result.get(value_col) is None or stale:
                    result[value_col], result[status_col] = None, "결측 / 센서 확인"
                else:
                    result[status_col] = "정상"
            if self.logger_process is not None:
                exit_code = self.logger_process.poll()
                result["logger_status"] = "수집 중" if exit_code is None else f"로거
종료 (코드 {exit_code})"
            else:
                result["logger_status"] = "기존 기록 연결"
        return result

def local_ip() -> str:
    try:
        result = subprocess.run(["hostname", "-I"], capture_output=True,
text=True, check=False)
        for value in result.stdout.split():
            if ":" not in value and not value.startswith("127."):
                return value
    except OSError:
        pass
    try:
        sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        sock.connect(("10.255.255.255", 1))

```

```

        value = sock.getsockname()[0]
        sock.close()
        return value
    except OSError:
        return "라즈베리파이_IP"

def resolve_logger(path_value: str) -> Path:
    path = Path(path_value)
    if not path.is_absolute() and not path.exists():
        candidate = Path(__file__).resolve().parent / path
        if candidate.exists():
            path = candidate
    if not path.exists():
        raise FileNotFoundError(f"로거 파일을 찾지 못했습니다: {path}")
    return path

def serve(bridge: Path, args: argparse.Namespace) -> None:
    monitor = SecurityMonitor(bridge, args)
    app = Flask(__name__)

    @app.get("/")
    def index():
        return HTML

    @app.get("/api")
    def api():
        return jsonify(monitor.get())

    @app.post("/api/security/arm")
    def arm():
        return jsonify(monitor.arm())

    @app.post("/api/security/disarm")
    def disarm():
        return jsonify(monitor.disarm())

    @app.post("/api/security/reset")
    def reset():
        return jsonify(monitor.reset_alert())

    print(f"보안 대시보드: http://{local_ip()}:{args.port}")
    print("대시보드에서 감시 시작을 누른 뒤 보정 시간 동안 기체를 건드리지 않는다.")
    app.run(host=args.host, port=args.port, threaded=True, use_reloader=False)

def main() -> None:
    args = parse_args()
    process: subprocess.Popen | None = None
    try:
        if args.bridge or args.manifest:
            bridge = Path(args.bridge or args.manifest).resolve()
            if not bridge.exists():
                raise FileNotFoundError(f"기록 파일을 찾지 못했습니다: {bridge}")
        else:
            logger = resolve_logger(args.logger).resolve()
            output = Path(args.output_dir).resolve()
            output.mkdir(parents=True, exist_ok=True)
            # 매 실행마다 고유 폴더를 만든다. 기존 CSV/메타데이터는 삭제하지 않는다.
            run_dir = Path(tempfile.mkdtemp(
                prefix=f"{args.condition}_{time.strftime('%Y%m%dT%H%M%S')}_",
                dir=output))
            bridge = run_dir / "current.json"

```

```

        command = [
            sys.executable, "-u", str(logger),
            "--condition", args.condition, "--run-dir", str(run_dir),
            "--sample-rate", str(args.sample_rate), "--range-g",
str(args.range_g),
            "--rotate-seconds", str(args.rotate_seconds),
            "--flush-seconds", str(args.flush_seconds),
            "--fsync-seconds", str(args.fsync_seconds),
            "--env-stale-seconds", str(args.env_stale_seconds),
        ]
        if args.no_dht:
            command.append("--no-dht")
        if args.no_surface:
            command.append("--no-surface")
        if args.surface_device:
            command.extend(["--surface-device", args.surface_device])
        process = subprocess.Popen(command)
        args.logger_process = process
        deadline = time.monotonic() + 45.0
        while not bridge.exists() and time.monotonic() < deadline:
            if process.poll() is not None:
                raise RuntimeError("로거가 먼저 종료되었습니다.
센서/라이브러리/디스크 오류를 확인하세요.")
                time.sleep(0.2)
            if not bridge.exists():
                raise RuntimeError("로거 시작 시간 초과: current.json이 생성되지
않았습니다.")
        print(f"[저장] 실행 폴더: {run_dir}")
        print(f"[저장] {args.rotate_seconds:g}초마다 새 csv. 기존 파일은
유지됩니다.")
        serve(bridge, args)
    finally:
        if process is not None and process.poll() is None:
            # 수정된 logger.py는 SIGTERM을 받아 큐를 비우고 flush/fsync 후 종료한다.
            process.terminate()
            try:
                process.wait(timeout=25.0)
            except subprocess.TimeoutExpired:
                print("[경고] 로거 종료 지연으로 강제 종료합니다. 마지막 파일을
점검하세요.")
                process.kill()
                process.wait()

if __name__ == "__main__":
    main()

```

logger.py

```

from __future__ import annotations

# 3차 실험: ADXL355 + DS18B20 표면온도 + DHT22 습도.
# 기본: 실행 시작 기준 3600초마다 CSV 교체. 수집 중 계속 기록/flush한다.
# temp_c는 호환성을 위해 DHT22 기온으로 유지하며 표면온도는 surface temp_c이다.
# DS18B20: logger4.py와 같은 1-Wire 설정(GPIO17). 태퍼 코드는 사용하지 않는다.
import argparse
import csv
import json
import math
import multiprocessing as mp
import os
import queue
import re

```

```

import signal
import tempfile
import threading
import time
from pathlib import Path
from typing import Final

ADXL355_ADDR_LOW: Final = 0x1D
ADXL355_ADDR_HIGH: Final = 0x53
REG_DEVID_AD: Final = 0x00
REG_TEMP2: Final = 0x06
REG_XDATA3: Final = 0x08
REG_FILTER: Final = 0x28
REG_RANGE: Final = 0x2C
REG_POWER_CTL: Final = 0x2D
REG_RESET: Final = 0x2F
EXPECTED_ID: Final = (0xAD, 0x1D, 0xED)
RANGE_TO_CODE = {2: 0x01, 4: 0x02, 8: 0x03}
RANGE_TO_SCALE_G_PER_LSB = {
    2: 2.048 / (2**19),
    4: 4.096 / (2**19),
    8: 8.192 / (2**19),
}
RANGE_TO_FULL_SCALE_G = {2: 2.048, 4: 4.096, 8: 8.192}
SAMPLE_RATE_TO_FILTER = {125: 0x05, 250: 0x04, 500: 0x03}
TEMP_BIAS_LSB: Final = 1852.0
TEMP_SLOPE_LSB_PER_C: Final = -9.05

def twos_complement(value: int, bits: int) -> int:
    sign_bit = 1 << (bits - 1)
    return value - (1 << bits) if value & sign_bit else value

def detect_address(bus: SMBus, requested: str) -> int:
    candidates = [ADXL355_ADDR_LOW, ADXL355_ADDR_HIGH] if requested == "auto" else [int(requested, 16)]
    for address in candidates:
        try:
            device_id = tuple(bus.read_i2c_block_data(address, REG_DEVID_AD, 3))
        except OSError:
            continue
        if device_id == EXPECTED_ID:
            return address
    raise RuntimeError("ADXL355를 찾지 못했습니다. i2cdetect -y 1에서 1d 또는 53을 확인해야 합니다.")

def configure_adxl355(bus: SMBus, address: int, range_g: int, sample_rate: int) -> None:
    bus.write_byte_data(address, REG_RESET, 0x52)
    time.sleep(0.1)
    bus.write_byte_data(address, REG_RANGE, 0x80 | RANGE_TO_CODE[range_g])
    bus.write_byte_data(address, REG_FILTER, SAMPLE_RATE_TO_FILTER[sample_rate])
    bus.write_byte_data(address, REG_POWER_CTL, 0x00)
    time.sleep(0.1)

def read_adxl_temperature_c(bus: SMBus, address: int) -> float:
    temp2, temp1 = bus.read_i2c_block_data(address, REG_TEMP2, 2)
    raw = ((temp2 & 0x0F) << 8) | temp1
    return 25.0 + (raw - TEMP_BIAS_LSB) / TEMP_SLOPE_LSB_PER_C

```

```

def read_acceleration_g(bus: SMBus, address: int, scale_g_per_lsb: float) ->
tuple[float, float, float]:
    data = bus.read_i2c_block_data(address, REG_XDATA3, 9)
    values: list[float] = []
    for index in range(0, 9, 3):
        raw20 = (data[index] << 12) | (data[index + 1] << 4) | (data[index + 2] >>
4)
        values.append(twos_complement(raw20, 20) * scale_g_per_lsb)
    return values[0], values[1], values[2]

def wait_until_ns(target_ns: int) -> None:
    while True:
        remaining_ns = target_ns - time.perf_counter_ns()
        if remaining_ns <= 0:
            return
        if remaining_ns > 2_000_000:
            time.sleep((remaining_ns - 1_000_000) / 1e9)
        else:
            time.sleep(0)

def unix_from_perf_ns(perf_ns: int, anchor_perf_ns: int, anchor_unix_s: float) ->
float:
    return anchor_unix_s + (perf_ns - anchor_perf_ns) / 1e9

CSV_COLUMNS = [
    "sample_index", "t_perf_ns", "t_unix_s", "scheduled_perf_ns",
    "lateness_us", "missed_slots_total",
    "temp_c", "humidity", "dht_age_s", "dht_temp_age_s",
    "surface_temp_c", "surface_age_s", "surface_ok", "humidity_ok",
    "adx1_temp_c", "ax_g", "ay_g", "az_g",
]
DS18B20_DEVICES_DIR = Path("/sys/bus/w1/devices")

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="SENTRY hourly CSV +
DS18B20/DHT22 logger")
    parser.add_argument("--condition", default="drift-check")
    parser.add_argument("--output-dir", default="data/check")
    parser.add_argument("--run-dir", help="대시보드가 생성한 빈 실행 폴더(내부용)")
    parser.add_argument("--bus", type=int, default=1)
    parser.add_argument("--address", choices=["auto", "0x1d", "0x53"],
default="auto")
    parser.add_argument("--sample-rate", type=int, choices=[125, 250, 500],
default=125)
    parser.add_argument("--range-g", type=int, choices=[2, 4, 8], default=8)
    parser.add_argument("--rotate-seconds", type=float, default=3600.0,
        help="실행 시작 기준 CSV 분할 간격. 기본 1시간")
    parser.add_argument("--flush-seconds", type=float, default=1.0)
    parser.add_argument("--fsync-seconds", type=float, default=10.0,
        help="OS에 파일 동기화 요청 간격. 정전 무손실 보장은 아님")
    parser.add_argument("--dht-interval", type=float, default=2.0)
    parser.add_argument("--surface-interval", type=float, default=2.0)
    parser.add_argument("--adx1-temp-interval", type=float, default=1.0)
    parser.add_argument("--env-stale-seconds", type=float, default=10.0,
        help="마지막 성공한 환경 측정값이 이 시간보다 오래되면 빈 칸")
    parser.add_argument("--surface-device", help="여러 DS18B20 사용 시 28-... 장치
ID")
    parser.add_argument("--no-dht", action="store_true")
    parser.add_argument("--no-surface", action="store_true")

```

```

args = parser.parse_args()
if not re.fullmatch(r"[\w.-]+", args.condition) or args.condition in {".",
".."}:
    parser.error("condition에는 문자, 숫자, _, -, .만 사용할 수 있습니다.")
for name in ("rotate_seconds", "flush_seconds", "fsync_seconds",
            "dht_interval", "surface_interval", "adx1_temp_interval",
"env_stale_seconds"):
    value = getattr(args, name)
    if not math.isfinite(value) or value <= 0:
        parser.error(f"{name}은 유효한 양수여야 합니다.")
if args.dht_interval < 2.0:
    parser.error("dht-interval은 2초 이상이어야 합니다.")
if args.surface_interval < 1.0:
    parser.error("surface-interval은 1초 이상이어야 합니다.")
return args

def create_run_dir(args: argparse.Namespace) -> Path:
    if args.run_dir:
        run = Path(args.run_dir).resolve()
        run.mkdir(parents=True, exist_ok=True)
        if any(run.iterdir()):
            raise FileExistsError(f"실행 폴더가 비어 있지 않습니다. 덮어쓰지 않습니다:
{run}")
        return run
    output = Path(args.output_dir).resolve()
    output.mkdir(parents=True, exist_ok=True)
    return Path(tempfile.mkdtemp(
        prefix=f"{args.condition}_{time.strftime('%Y%m%dT%H%M%S')}_{", dir=output))

def sync_directory(path: Path) -> None:
    fd = os.open(str(path), os.O_RDONLY | os.O_DIRECTORY)
    try:
        os.fsync(fd)
    finally:
        os.close(fd)

def atomic_json(path: Path, value: dict) -> None:
    # 동일 파일시스템에서 원자적 교체: 대시보드가 반쪽짜리 JSON을 읽지 않게 한다.
    fd, temporary = tempfile.mkstemp(prefix="." + path.name, dir=path.parent)
    try:
        with os.fdopen(fd, "w", encoding="utf-8") as file:
            json.dump(value, file, ensure_ascii=False, indent=2, allow_nan=False)
            file.flush()
            os.fsync(file.fileno())
        os.replace(temporary, path)
        sync_directory(path.parent)
    finally:
        if os.path.exists(temporary):
            os.unlink(temporary)

def find_ds18b20_device(device_id: str | None = None) -> Path:
    if device_id:
        if not re.fullmatch(r"28-[0-9a-fA-F]+", device_id):
            raise ValueError("surface-device에는 28-... 형식의 장치 ID를 지정하세요.")
        candidates = [DS18B20_DEVICES_DIR / device_id / "w1_slave"]
        candidates = [path for path in candidates if path.exists()]
    else:
        candidates = sorted(DS18B20_DEVICES_DIR.glob("28-*/w1_slave"))
    if not candidates:
        raise RuntimeError(

```

```

        "DS18B20 없음: /boot/firmware/config.txt의 "
        "dtoverlay=w1-gpio,gpiopin=17 설정과 재부팅, 28-* 장치를 확인하세요.")
    if len(candidates) > 1:
        raise RuntimeError("DS18B20이 여러 개입니다. --surface-device 28-... 로
지정하세요.")
    return candidates[0]

def read_ds18b20_c(path: Path) -> float:
    # logger4.py의 CRC 판독 방식. 블로킹 판독은 별도 프로세스에서만 수행.
    try:
        lines = path.read_text().strip().splitlines()
        if len(lines) < 2 or not lines[0].strip().endswith("YES"):
            return math.nan
        value = int(lines[1].partition("t=")[2].strip()) / 1000.0
    except (OSError, ValueError):
        return math.nan
    # logger4.py와 동일하게 파워온 기본값 85°C는 무효로 처리한다.
    return value if -55 <= value <= 125 and value != 85.0 else math.nan

def publish_sensor(output_queue: mp.Queue, item: tuple) -> None:
    try:
        output_queue.put_nowait(item)
    except queue.Full:
        # 최신 측정만 필요. Queue feeder와의 경합 시 다음 측정에서 다시 전달.
        try:
            output_queue.get_nowait()
        except queue.Empty:
            pass
        try:
            output_queue.put_nowait(item)
        except queue.Full:
            pass

def dht_worker(output_queue: mp.Queue, stop_event: mp.Event, interval_s: float) ->
None:
    import adafruit_dht
    import board
    dht = adafruit_dht.DHT22(board.D4)
    try:
        while not stop_event.is_set():
            temp, hum = math.nan, math.nan
            try:
                t, h = dht.temperature, dht.humidity
                if t is not None and math.isfinite(float(t)):
                    temp = float(t)
                if h is not None and math.isfinite(float(h)) and 0 <= float(h) <=
100:
                    hum = float(h)
            except (RuntimeError, OSError):
                pass
            publish_sensor(output_queue, (time.perf_counter_ns(), temp, hum))
            stop_event.wait(interval_s)
    finally:
        dht.exit()

def surface_worker(output_queue: mp.Queue, stop_event: mp.Event,
                    interval_s: float, device_path: str) -> None:
    while not stop_event.is_set():
        value = read_ds18b20_c(Path(device_path))
        publish_sensor(output_queue, (time.perf_counter_ns(), value))

```

```

        stop_event.wait(interval_s)

class SensorCache:
    def __init__(self) -> None:
        self.values: dict[str, tuple[int, float]] = {}

    def update(self, name: str, measured_ns: int, value: float) -> None:
        # 실패한 측정 시도가 아닌, 성공한 값에 대해서만 시각을 갱신한다.
        if math.isfinite(value):
            self.values[name] = (measured_ns, value)

    def read(self, name: str, now_ns: int, max_age_s: float) -> tuple[float, float, bool]:
        if name not in self.values:
            return math.nan, math.nan, False
        measured_ns, value = self.values[name]
        age = max(0.0, (now_ns - measured_ns) / 1e9)
        valid = age <= max_age_s
        return (value if valid else math.nan), age, valid

def format_optional(value: float, digits: int = 3) -> str:
    return "" if not math.isfinite(value) else f"{value:.{digits}f}"

class RotatingCSVWriter:
    """Bounded queue separates CSV flush/fsync/rotation from the sensor loop."""
    def __init__(self, run_dir: Path, metadata: dict, *, queue_size: int = 1000):
        self.run_dir = run_dir
        self.metadata = metadata
        self.manifest = run_dir / "current.json"
        self.pending: queue.Queue = queue.Queue(maxsize=queue_size)
        self.stop_event = threading.Event()
        self.ready = threading.Event()
        self.error: BaseException | None = None
        self.saved_rows = 0
        self.file = None
        self.writer = None
        self.current_path: Path | None = None
        self.previous_path: Path | None = None
        self.part = 0
        self.thread = threading.Thread(target=self._run, name="csv-writer",
daemon=True)
        self.thread.start()
        if not self.ready.wait(30.0):
            self.stop_event.set()
            raise TimeoutError("CSV 저장 스레드 시작 시간 초과")
        self.check()

    def check(self) -> None:
        if self.error is not None:
            raise RuntimeError(f"CSV 저장 실패: {self.error}") from self.error

    def put(self, row: list, sample_ns: int) -> None:
        self.check()
        try:
            self.pending.put_nowait((row, sample_ns))
        except queue.Full as exc:
            raise RuntimeError("디스크 기록이 밀려 저장 큐가 가득 찼습니다. 수집을 중단합니다.") from exc

    def publish(self, status: str) -> None:
        atomic_json(self.manifest, {

```

```

        **self.metadata,
        "current_csv": self.current_path.name if self.current_path else None,
        "previous_csv": self.previous_path.name if self.previous_path else
None,
        "part": self.part, "status": status, "saved_rows": self.saved_rows,
        "manifest_updated_unix_s": time.time(),
    })

def _sync(self) -> None:
    self.file.flush()
    os.fsync(self.file.fileno())

def _open_part(self, part: int) -> None:
    if self.file is not None:
        self._sync()
        self.file.close()
    self.previous_path = self.current_path
    self.part = part
    self.current_path = self.run_dir / f"part_{part:06d}_bridge_log.csv"
    # x 모드: 기존 CSV는 어떤 경우에도 덮어쓰지 않는다.
    self.file = self.current_path.open("x", newline="", encoding="utf-8",
                                       buffering=1024 * 1024)

    self.writer = csv.writer(self.file)
    self.writer.writerow(CSV_COLUMNS)
    self._sync()
    atomic_json(self.current_path.with_name(f"part_{part:06d}_metadata.json"),
{
        **self.metadata, "part": part,
        "segment_start_elapsed_s": (part - 1) *
self.metadata["rotate_seconds"],
    })
    self.publish("running")
    print(f"[저장] 현재 CSV: {self.current_path}", flush=True)

def _run(self) -> None:
    try:
        self._open_part(1)
        self.ready.set()
        next_flush = time.monotonic() + self.metadata["flush_seconds"]
        next_sync = time.monotonic() + self.metadata["fsync_seconds"]
        rotate_ns = int(self.metadata["rotate_seconds"] * 1e9)
        while not self.stop_event.is_set() or not self.pending.empty():
            try:
                row, sample_ns = self.pending.get(timeout=0.1)
            except queue.Empty:
                row = None
            if row is not None:
                elapsed = max(0, sample_ns - self.metadata["anchor_perf_ns"])
                part = 1 + elapsed // rotate_ns
                if part != self.part:
                    self._open_part(part)
                    self.writer.writerow(row)
                    self.saved_rows += 1
                now = time.monotonic()
                if now >= next_flush:
                    self.file.flush()
                    next_flush = now + self.metadata["flush_seconds"]
                if now >= next_sync:
                    self._sync()
                    self.publish("running")
                    next_sync = now + self.metadata["fsync_seconds"]
                self._sync()
                self.publish("stopped")
            except BaseException as exc:

```

```

        self.error = exc
        print(f"[저장 오류] {exc}", flush=True)
    finally:
        self.ready.set()
        if self.file is not None:
            try:
                self.file.close()
            except OSError as exc:
                if self.error is None:
                    self.error = exc

def close(self) -> None:
    self.stop_event.set()
    self.thread.join(timeout=15.0)
    if self.thread.is_alive():
        raise TimeoutError("CSV 마감이 지연되고 있습니다. 디스크 상태를 확인하세요.")
    self.check()

def main() -> None:
    args = parse_args()
    # 하드웨어 모듈은 실제 수집을 시작할 때만 필요하다.
    from smbus2 import SMBus
    run_dir = create_run_dir(args)
    stop = threading.Event()
    for signum in (signal.SIGINT, signal.SIGTERM):
        signal.signal(signum, lambda _signum, _frame: stop.set())
    children = []
    queues = {}
    writer = None
    cache = SensorCache()
    total_samples = missed_slots = read_errors = 0
    clip_counts = {"ax": 0, "ay": 0, "az": 0}
    latest_adxl_temp = math.nan
    last_adxl_temp_ns = 0
    sample_interval_ns = int(1e9 / args.sample_rate)
    scale = RANGE_TO_SCALE_G_PER_LSB[args.range_g]
    clip_limit = 0.995 * RANGE_TO_FULL_SCALE_G[args.range_g]
    surface_path = None
    try:
        if not args.no_surface:
            surface_path = find_ds18b20_device(args.surface_device)
        context = mp.get_context("spawn")
        specs = []
        if not args.no_dht:
            specs.append(("dht", dht_worker, (args.dht_interval,)))
        if surface_path is not None:
            specs.append(("surface", surface_worker, (args.surface_interval,
str(surface_path))))
        for name, target, extra in specs:
            sensor_queue = context.Queue(maxsize=4)
            sensor_stop = context.Event()
            process = context.Process(target=target, args=(sensor_queue,
sensor_stop, *extra),
                                   daemon=True)

            process.start()
            queues[name] = sensor_queue
            children.append((process, sensor_stop, sensor_queue))

    with SMBus(args.bus) as bus:
        address = detect_address(bus, args.address)
        configure_adxl355(bus, address, args.range_g, args.sample_rate)
        anchor_ns, anchor_unix = time.perf_counter_ns(), time.time()
        metadata = {

```

```

        "schema_version": 3, "condition": args.condition,
        "run_id": run_dir.name, "sample_rate_hz": args.sample_rate,
        "range_g": args.range_g, "address_hex": f"0x{address:02X}",
        "filter_register_hex":
f"0x{SAMPLE_RATE_TO_FILTER[args.sample_rate]:02X}",
        "anchor_perf_ns": anchor_ns, "start_unix_s": anchor_unix,
        "rotate_seconds": args.rotate_seconds,
        "flush_seconds": args.flush_seconds, "fsync_seconds":
args.fsync_seconds,
        "env_stale_seconds": args.env_stale_seconds,
        "temperature_source": "DS18B20", "humidity_source": "DHT22",
        "surface_device": str(surface_path) if surface_path else None,
        "surface_enabled": not args.no_surface, "dht_enabled": not
args.no_dht,
        "temp_c_meaning": "DHT22 air temperature (reference only)",
        "dht_age_s_meaning": "age since last successful humidity reading",
    }
    atomic_json(run_dir / "session_metadata.json", metadata)
    writer = RotatingCSVWriter(run_dir, metadata,
queue_size=args.sample_rate * 5)
    print(f"[수집] {args.sample_rate} Hz / ±{args.range_g} g /
{args.rotate_seconds:g}초 분할")
    print(f"[수집] 실행 폴더: {run_dir}\n[수집] Ctrl+C로 안전 종료",
flush=True)
    next_sample_ns = time.perf_counter_ns()
    while not stop.is_set():
        writer.check()
        wait_until_ns(next_sample_ns)
        if stop.is_set():
            break
        now_ns = time.perf_counter_ns()
        if now_ns - next_sample_ns >= sample_interval_ns:
            skipped = (now_ns - next_sample_ns) // sample_interval_ns
            missed_slots += skipped
            next_sample_ns += skipped * sample_interval_ns
        scheduled_ns = next_sample_ns
        for name, sensor_queue in queues.items():
            try:
                while True:
                    item = sensor_queue.get_nowait()
                    if name == "dht":
                        measured_ns, temp, hum = item
                        cache.update("air", measured_ns, temp)
                        cache.update("humidity", measured_ns, hum)
                    else:
                        measured_ns, surface = item
                        cache.update("surface", measured_ns, surface)
            except queue.Empty:
                pass
        if now_ns - last_adxl_temp_ns >= int(args.adxl_temp_interval *
1e9):
            try:
                latest_adxl_temp = read_adxl_temperature_c(bus, address)
            except OSError:
                latest_adxl_temp = math.nan
            last_adxl_temp_ns = now_ns
            read_start = time.perf_counter_ns()
            try:
                ax, ay, az = read_acceleration_g(bus, address, scale)
            except OSError:
                read_errors += 1
                next_sample_ns += sample_interval_ns
                continue
            sample_ns = (read_start + time.perf_counter_ns()) // 2

```

```

        for axis, value in (("ax", ax), ("ay", ay), ("az", az)):
            if abs(value) >= clip_limit:
                clip_counts[axis] += 1
            air, air_age, _ = cache.read("air", sample_ns,
args.env_stale_seconds)
            hum, hum_age, hum_ok = cache.read("humidity", sample_ns,
args.env_stale_seconds)
            surface, surface_age, surface_ok = cache.read("surface",
sample_ns, args.env_stale_seconds)
            row = [
                total_samples, sample_ns,
                f"{unix_from_perf_ns(sample_ns, anchor_ns, anchor_unix):.9f}",
                scheduled_ns, f"{{sample_ns - scheduled_ns} / 1000:.3f}",
missed_slots,
                format_optional(air), format_optional(hum),
format_optional(hum_age),
                format_optional(air_age), format_optional(surface),
format_optional(surface_age),
                int(surface_ok), int(hum_ok),
format_optional(latest_adxl_temp),
                f"{ax:.8f}", f"{ay:.8f}", f"{az:.8f}",
            ]
            writer.put(row, sample_ns)
            total_samples += 1
            next_sample_ns += sample_interval_ns

    finally:
        stop.set()
        for process, sensor_stop, sensor_queue in children:
            sensor_stop.set()
        try:
            if writer is not None:
                writer.close()
        finally:
            for process, sensor_stop, sensor_queue in children:
                process.join(timeout=3.0)
                if process.is_alive():
                    process.terminate()
                    process.join(timeout=2.0)
                sensor_queue.close()
            qc = {
                "queued_samples": total_samples,
                "saved_rows": writer.saved_rows if writer else 0,
                "missed_slots": missed_slots, "read_errors": read_errors,
                "clip_counts": clip_counts,
            }
            try:
                atomic_json(run_dir / "qc_summary.json", qc)
            except OSError as exc:
                print(f"[QC 저장 실패] {exc}", flush=True)
            print(f"[종료/QC] {qc}", flush=True)

if __name__ == "__main__":
    main()

```